NoiseSentinel – Portable IoT Stick for Detecting Modified Automobile Silencers

Final Year Project

2022-2026

A project submitted in partial fulfillment of the degree of

BS in Computer Science

Submitted to

Dr. Abdul Hameed

Department of Computer Science

Faculty of Computing & Artificial Intelligence (FCAI)

Air University, Islamabad

Type (Nature of project)	[✓] Development [] R&D
Area of specialization	[✓] WebApp [✓] Mobile App [] AI based [✓] Embedded System
FYP ID	

Project Group Members

Sr.#	Reg. #	Student Name	Email ID	*Signature
(I)	221073	Muhammad Moiz	221073@students.au.edu.pk	
(ii)	221089	Ifsan Imran	221089@students.au.edu.pk	
(iii)	221336	Mishkat Fatima	221336@students.au.edu.pk	

*The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to work of others

Plagiarism Certificate

This is to certify that, I Mishkat Fatima S/D of Muhammad Shabbir, group leader of FYP under registration no 221336 Computer Sciences Department, Air University. I declare that my FYP report is checked by my supervisor.

Date: _____ Name of Group Leader: Mishkat Fatima Signature:_____

Name of Supervisor: Dr. Abdul Hameed

Designation: HOD (Kharian Campus)

Signature: _____

HOD: Dr. Ashfaq Ahmad

Signature: _____

Noise Sentinel

Change Record

Author(s)	Version	Date	Notes	Supervisor's Signature

APPROVAL

PROJECT SUPERVISOR

Name: Dr. Abdul Hameed

Date: _____ Signature: _____

PROJECT MANAGER

Date: _____ Signature: _____

CHAIR DEPARTMENT

Date: _____ Signature: _____

Dedication

This work is dedicated to our beloved parents whose unconditional love, countless sacrifices and unwavering prayers have been our greatest strength throughout this journey. The groundwork they laid for us enabled us to overcome every challenge and achieve every milestone every late night.

We also dedicate this work to the people of Pakistan, especially the urban folks, who suffer silently due to uncontrolled noise and vehicular emissions, which affect their quality of life. We hope that NoiseSentinel is a significant move in the direction of responsible and technologically advanced policing practices that benefit society.

Acknowledgements

Firstly, we would like to give our special thanks to our project supervisor Dr Abdul Hameed, whose guidance, patience and technical understanding played a pivotal role in all aspects of this project. His valuable guidance supported us in making challenging architecture decisions and his encouragement sustained us through the most challenging periods of development.

The authors also express their special thanks to all the faculty members and administration of Computer Science Department, Air University, Islamabad for giving us a solid academic base and resources to complete this project.

We are deeply grateful to our parents/families whose unconditional love, prayers and sacrifices strengthened us and gave us our peace of mind when we wanted to give up. They always believed in us, when we didn't.

Then we give thanks to one another. This project was created via real collaboration, from the system architecture to deployment on a live production server. Debugging the firmware at odd hours, calibrating sensors, and taking multiple tries on API design and pushing through integration challenges would not have happened without the trust and commitment of each team member.

Executive Summary

Violations of noise and vehicular emission standard are an emerging public health and environmental issue in the cities of Pakistan. Although there are well-established legal criteria, the implementation is still, for the most part, manual, disjointed, and challenging to maintain at scale. They are dependent on subjective judgements, and documentation is easily lost or manipulated and coordination is lacking among officers, administrative authorities, and the judiciary. The outcome is a system that is slow, opaque and easily circumvented.

NoiseSentinel is a solution that tackles the issue from beginning to end with an integrated multi-platform enforcement and case management system consisting of three connected components: an ESP32 based IoT sensing device, a React Native case management mobile app for police officers, a case management webapp driven React web portal.

The IoT device is a mobile, BLE sensor unit that is carried by field officers. It is equipped with a calibrated electret microphone for sound level measurements in decibels (dBA) and MQ-series gas sensors which measure the emissions of vehicles using Bluetooth Low Energy.

Android mobile application created and is for police officers only. Officers use it to pair with the IoT device, for noise and emission tests, to check vehicles via plate number and accused persons via CNIC, to attach photographic evidence and to issue digital challans.

NET, is available via a Web API on ASP.NET Core (The system's backbone is the network (NET 8). It presents a well-designed REST API that's used by the mobile app and the web portal.

The web portal has 6 user roles Administrator, Police Officer, Station Authority, Court Authority, Judge and Public. Administrators control system users and IoT device registration. Challans are looked into by Station Authorities and further escalated for Judicial Processing of valid cases. Court Authorities determine when hearings are to be held and appoint judges. Verdicts and close cases are recorded by judges. The public module enables the accused to check the status of their case by entering the vehicle number and CNIC, and to pay the challan through the bank transfer procedure listed on the screen.

NoiseSentinel brings the enforcement process away from a manual paper-based process to a completely electronic, tamper-proof and judicially-traceable process. It provides objectivity via calibrated hardware, accountability via role-based access and digital records, and accessibility via live production deployment that is accessible to all stakeholders.

Table of Contents

List of Tables

List of Figures

Chapter 1
Introduction & Background

Introduction

This chapter presents the NoiseSentinel project and offers a detailed description of the problem domain and the rationale behind the development of an automated multi-platform enforcement system for violations of vehicular noise and emissions. It describes the context of the problem, the existing gaps in current enforcement, the main purpose and goals of the system, existing solutions and their drawbacks, and a proposed solution that fills the gaps identified. The chapter ends with the project plan, which is a summary of the work breakdown structure, the roles of the team, and a summary of the rest of this report.

1.1 Background

In the urban centres of Pakistan noise pollution and vehicular exhaust emissions have become serious environmental and public health concerns. With more and more vehicles on the roads of metropolitan areas such as Islamabad, Rawalpindi, Lahore etc, the cumulative impact of modified silencers, unmaintained engines and unchecked exhausts is a cause of high noise levels along with the alarming levels of Carbon Monoxide (CO), Hydrocarbons (HC) and Nitrogen Oxides (NOx) in the air. Yes, there are regulatory frameworks, but they have not been effective — Pakistan's Motor Vehicle Rules include noise limits, as do the EURO-2 emission limits for vehicle exhaust. In the past, traffic cops have used handheld decibel meters to check the noise levels of cars suspected of noise pollution violations, and then manually handed out paper citations, or challans. Emission testing, if they have it, is also subjectively done by hand. There is no continuous, verifiable data trail with this approach. Readings are not securely attached to specific cars, evidence is not digitally retained and administrative processes to convert a citation into a formal legal matter are slow, paper based, and lose data. This creates a system of enforcement that is easy to challenge, challenging to maintain on a large scale, and mostly divorced from the judicial process it was intended to serve.

1.2 Motivations and Challenges

The main driver for developing NoiseSentinel is the lack of a clear and growing disconnect between paper and reality when it comes to enforcing the law. Officers don't have the tools to obtain objective and defensible evidence. There are poor coordination and fragmentation of administration systems. The judiciary then gets cases lacking supporting documentation or with

incomplete documentation. These problems all exacerbate each other and result in low prosecution rates and little deterrent effect.

Here are some of the challenges facing this landscape:

- **Violation detection is subjectivity.** If a police officer has given a challan for noise or emission violation as per manual reading, then the accused can challenge it on the ground that the equipment used for measuring was not calibrated, the reading was incorrect, or the officer was measuring the wrong vehicle. A direct and automatic link between the sensor data and the vehicle and time of the detection is hard to prove if a system is not in place.
- **No evidence record is available.** Current paper and low-tech electronic ticketing systems consider any sensor data as a human-keyed number on a form. At the time of capture, there is no mechanism to directly link with the hardware measurement to the digital record. The disassociation makes it possible to doubt any reading as a possible transcription error or a malicious manipulation.
- **Administrative separation of enforcement and judiciary.** Under the law in Pakistan, traffic violations fall into two types: non-cognizable and cognizable. Non-cognizable traffic offences are those that can be punished by a fine, while cognizable traffic offences involve registering a formal First Information Report (FIR) and take the matter to court. Today, escalation of challan to FIR and then to a court case involves hard copy data transfer from police station to court, manual re-entry of data and coordination between several non-integrated systems. This results in many cognizable offences not being prosecuted and in serious delays with high administrative overhead.
- **Lack of a digital work chain in coordination among stakeholders.** Police officers, station authorities, court officers and judges currently do not have a digital platform to share. Data collected by the officials at the point of enforcement does not automatically go to administrative or court bodies. Every hand-off is manual and provides opportunities for data loss, manipulation, and delay.

1.3 Goals and Objectives

The long-term objective of the NoiseSentinel project is to create an end-to-end, digitized enforcement pipeline to oversee the entire life cycle of a vehicular noise and emission violation

from the time the measurement is collected in the field to the point where a final decision is made in court, using a multi-platform system approach.

This is achieved through four key objectives:

- **Objective 1: Sensor to mobile enforcement integration using IoT.** The system is composed of an ESP32 IoT sensing device custom-built by the students and a mobile application to be used by the field officers, developed with the React Native framework for mobile development. The sound level is measured in decibels (dBA) by a calibrated electret microphone, while CO, HC and NOx emissions from vehicles are measured by MQ-series gas sensors. The device and mobile app communicate via Bluetooth Low Energy (BLE) for real-time, wireless data sharing without manual data entry.
- **Objective 2: Violations can be classified on device.** The ESP32 firmware also has a classification engine which compares the readings captured with the regulatory requirements of Pakistan and tags each with the type of violation and the severity of the violation. Thresholds for noise readings are from compliant (80 dBA or less) to severe violation (95 dBA or more). The evaluation for emissions is based on the EURO-2 standard for cars and motorcycles. This classification is made on the device itself, and is shared as part of the sensor response as an objective, hardware-based classification in addition to the officer's enforcement action.
- **Objective 3: Digitalization of challans connected with evidence.** The mobile application enables officers to generate formal digital challans with evidence in digital format which includes data by sensor, photos, information of the accused person based on CNIC, and information of the vehicle based on vehicle number. An Emission Report is auto created from IoT readings and is connected with the challan and a record is carried along with the case at every stage of the legal process.
- **Objective 4: Multi-Stakeholder Case Management Based on Objectives.** The system follows a strict Role Based Access Control (RBAC) mechanism having 06 user roles – Administrator, Police Officer, Station Authority, Court Authority, Judge and Public. Each role is only concerned with the functionality that is related to their role in the enforcement and judicial workflow. Challans are reviewed and approved by Station Authorities who can also upgrade cognizable offences to full fledged FIRs. Hearings are set and judges are appointed by Court Authorities. Judges record verdicts. This

separation of powers also helps maintain integrity throughout a case, as no actor can have the whole case life cycle.

1.4 Literature Review / Existing Solutions

Looking at the current solutions for traffic and environmental enforcement, there are several types of solutions that aim at solving a part of the problem; there is no comprehensive solution that offers a complete and connected workflow.

- **Independent Sound Level Meters (SLMs).** The most common tool in current use, handheld SLMs are scientifically accurate instruments for measuring decibel levels. But there's no interaction between them. They lack network connectivity, can't tie a reading to a vehicle registration and must have the officer manually record and transcribe the value into a paper citation. Once the reading is written down by the officer, the direct link from the instrument to the record is severed, and the reading would be legally challengeable.
- **Smart City Acoustic Camera Systems.** Stationary acoustic camera installations are located at the higher end of the spectrum and when installed with Automatic Number Plate Recognition (ANPR), can automatically identify and record the number plates of loud vehicles passing by. The systems are technically feasible and have been implemented in different European cities. They, however, need permanent infrastructure, are very costly and cannot be moved at all for widespread use in developing countries. They are not available for dynamic use in different locations with officers led enforcement.
- **Mobile Applications for generic e-Challan.** Some law enforcement agencies have launched apps for smartphones that enable cops to fine a car by simply inputting the registration number of the car. These are helpful administrative tools, but are simply software based. They do not interact with any sensors in the environment. Even if an officer uses an e-Challan app for noise violation, the officer is required to manually read a meter and type the value into the app, and is subject to the same human error and evidentiary conflict issues as paper systems.
- **Judicial Case Management Systems (CMS).** The pre-existing court management systems provide the facility of tracking FIRs, hearing schedules and verdicts. These systems do their job in the judiciary and are totally divorced from field enforcement.

The police need to provide the evidence to court authorities either manually or physically before they can start a case. No automated connection between violation detection and formal legal procedure.

1.5 Gap Analysis

However, the primary gap in the current landscape is the lack of an end-to-end process from hardware level evidence collection to administrative processing and judicial oversight, all in a single integrated platform.

- **The hardware-software disconnect.** There is no current solution that natively connects physical sensing hardware to digital enforcement software. Officers have to use two separate tools: a sensor and a ticketing app, and there is no direct integration between the two. This is the main reason for the lack of evidentiary vulnerability in enforcement.
- **There is no continuous evidence record.** Currently in place e-ticketing systems hold evidence in the form of individually entered data text fields or uploaded images to storage with no cryptographic or structural tie-in between the hardware reader and the database record. The reading and record are two distinct objects. NoiseSentinel's solution overcomes this by creating an Emission Report directly from the IoT device response, and binding it structurally to the challan at the time of creation, ensuring that the sensor data and the citation exist as a single connected record, and not as two separate records.
- **The legal workflow bottleneck is the legal process that consumes the most time.** No current application exists to automatically classify the severity of a traffic citation and add it to the right legal process. The whole process of escalating a field challan to a station level FIR to court hearing is now spread across several disjointed processes and manual hand-offs. This fragmentation leads to both high administrative costs and long delays, and low prosecution success rates for environmental violations.
- **Emission enforcement neglect.** Current noise enforcement measures are only based on noise levels. Vehicular exhaust emission monitoring CO, HC, NOx is considered as a domain which is totally different from the other domains with different processes. NoiseSentinel is the first to integrate noise and emission monitoring and control in a unified workflow, a single IoT sensor, a single mobile app and a single backend system.

1.6 Proposed Solution

NoiseSentinel is presented as a full-motion, multi-platform enforcement and case management solution consisting of four interrelated elements, which together create an unbroken digital workflow from the field measurement to a judicial resolution.

- **IoT Sensing Device (ESP32).** Portable battery-powered device carried by field officers. It is equipped with a calibrated electret microphone for noise measurement, and MQ-series gas sensors for emission analysis. The device uses BLE to communicate with the officer's mobile application via a JSON command/response protocol. Once the officer starts a test, all the necessary measurements are taken by the device, violation type is automatically classified on-device, and the entire result is automatically sent to the mobile app wirelessly.
- **Mobile Application (React Native / Android).** Developed in React Native and specifically for police officers using Expo. The application includes the functioning of BLE devices pairing, initiation of testing, real-time monitoring of testing progress during emission warm-up and sampling, vehicle and accused person look-up, photo capturing and digital challan submission. All API interactions are made over HTTPS and authentication tokens are encrypted in the device storage.
- **ASP.NET Core Web API (.NET 8).** It is the heart of the system where the authentication process for users is managed using JWT, email verification using OTP, management of challan and emission reports, IoT device registration, FIR management, court case scheduling and recording the verdict. The backend uses the RBAC (role-based access control) architecture to control which actions each of the six user roles can take.
- **Web Portal (React + Vite + TypeScript).** It offers role specific dashboards for Administrators, Station Authorities, Court Authorities and Judges and has a public facing module, which allows accused persons to check the status of their case by vehicle number and CNIC, view any challans issued in their case and follow the instructions on bank transfers to make a payment.
- It's installed on a DigitalOcean droplet with Docker Compose and added to Cloudflare DNS and uses Caddy for automatic HTTPS. Production domain: noisesentinel.tech.

Database: Azure SQL Server. Transaction emails (OTP verification code): SMTP2GO with the system's own domain.

- The architecture has been designed with a clear three-tier separation: the presentation layer includes the mobile application and web portal, the business logic layer is the Web API, and the data layer is the Azure SQL database that is accessed using the Entity Framework Core. This separation assures maintainability, security, and the ability to scale individual components.

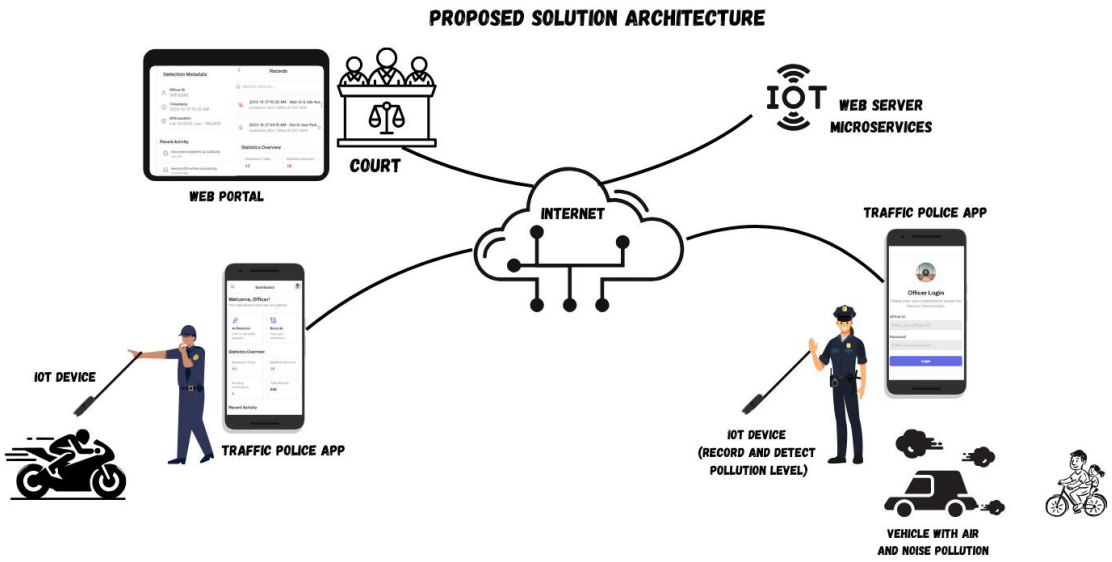


Figure 1: Proposed Solution Architecture

1.7 Project Plan

The project plan outlines the project's step-by-step design, development, testing and deployment of the NoiseSentinel platform throughout the academic year. It sets the structural division of development tasks, defines responsibilities of the team members and gives a time frame, where all the deliverables were completed before the final defence evaluation.

1.7.1 Work Breakdown Structure

The WBS breaks down the work for the NoiseSentinel project into five successive phases, each having well-defined deliverables.

Phase	Name	Description
Phase 1	Project Initiation and Requirements	Literature Review, Problem Domain Analysis, Requirements Gathering from Law Enforcement and Judicial Areas and Finalising the project proposal and scope.
Phase 2	System Design	Three-tier system architecture, mobile application UI/UX wireframing, REST API endpoint design, Entity-Relationship Diagram (ERD) development and BLE protocol specification.
Phase 3	Implementation and Integration	Core development phase – ESP32 firmware development and sensor calibration, React Native mobile application development, ASP.NET Core Web API building, web portal based on React building, integration of BLE between the IoT device and mobile application.
Phase 4	Testing and Quality Assurance	Unit testing of sensor evaluation logic and classification algorithms, API integration testing, role-based access control security testing, end to end workflow validation for all user roles, performance testing under concurrent load.
Phase 5	Deployment and Documentation	Cloud deployment using Docker on DigitalOcean, domain and HTTPS configuration using Cloudflare and Caddy, setup of Azure SQL Server, SMTP2GO email integration, generation of mobile application builds, and compilation of final project documentation.

Table 1: Work breakdown structure for NoiseSentinel

1.7.2 Roles & Responsibility Matrix

The work was assigned to team members based on their technical skills and areas of domain expertise. Each team member has a primary area of responsibility as outlined below.

Point Person	Responsibility
Muhammad Moiz (221073)	**IoT Device and Connectivity** — ESP32 firmware development, electret microphone and MQ-sensors calibration, violation classification logic in the device, BLE server implementation and design of communication protocol using JSON.
Ifsan Imran (221089)	**Mobile Application** — Engineering of the officer application (React Native), pairing with BLE devices, initiating IoT testing, handling results, challan creation workflow, vehicle and accused search, photograph evidence capture, and secure API communication.
Mishkat Fatma (221336)	**Web Portal and Backend** — Development of the ASP.NET Core Web API, Database Schema Design, Entity Framework Core integration, Role Based Access Control implementation and the Web Portal with all administrative, station authority, court authority and judge dashboards using React.

Table 2: Roles and Responsibilities Matrix

1.7.3 Gantt Chart

The Gantt Chart shows each phase and the tasks that make up each phase with start and end dates, concurrent workstreams (mobile and backend for example) and critical relationships between the components. The chart showed that all big deliverables were ahead of schedule to be completed before the final defence.

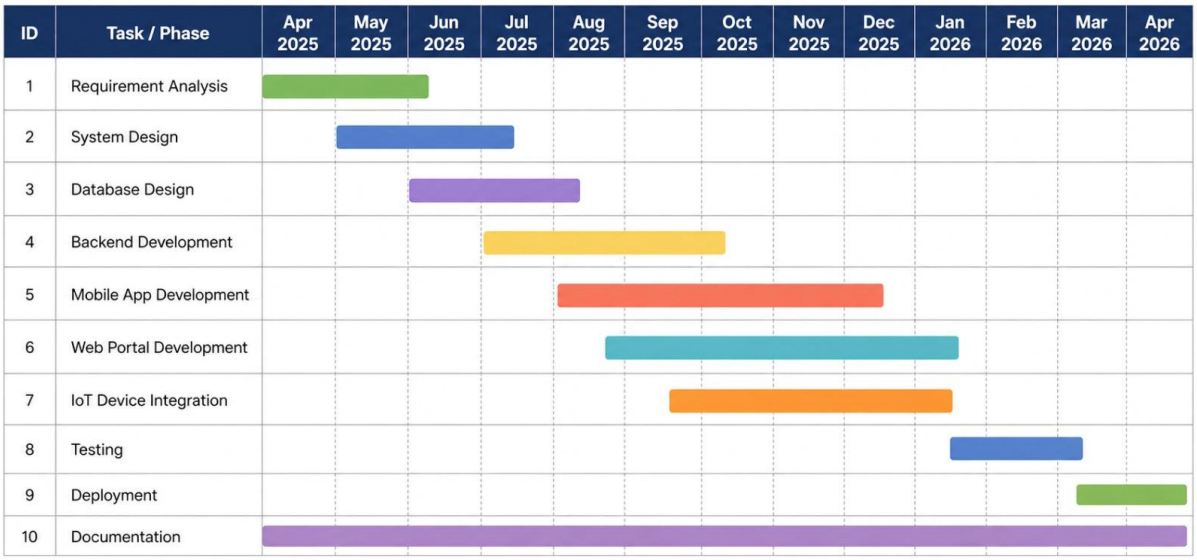


Figure 2: Gantt Chart for NoiseSentinel

1.8 Report Outline

The rest of this document is structured and technical about the NoiseSentinel system. Chapter 2 discusses Software Requirements Specification, in which functional requirements for each of the user roles are specified and non-functional requirements such as security, performance and reliability are discussed. Chapter 3 provides information about the System Architecture and Design which includes the ERD, system architecture diagrams, and the workflow logic for each major process. Chapter 4 covers the Implementation, and the technical decisions that were made throughout the ESP32 firmware, React Native application, ASP.NET Core API and React web portal. Chapter 5 discusses Testing and Quality Assurance, and the test cases and results are given to validate the system based on its functional and non-functional requirements. In Chapter 6, the Conclusions are presented, which summarize the project results and suggest directions for future improvements.

Chapter 2
Software Requirements
Specifications

2.1 Introduction

2.1.1 Purpose

The following Software Requirements Specification (SRS) is for the NoiseSentinel platform, version 1.0. This document describes the entire product that includes three main subsystems: the mobile app from the ground police officers using React Native, the .NET 8.0 Web API backend serving as the central processing unit and the Web Portal used by judicial and administration authorities. It outlines the functional, non-functional and interface requirements that must be met to establish an end-to-end digital evidence chain for traffic noise enforcement.

2.1.2 Document Conventions

This SRS contains all the elements of a software engineering document, with clear and structured formatting. Content is organized hierarchically to make it easier to read using headings and subheadings. In particular, the typographic style is used for all main titles (14 pt Times New Roman (Bold)) throughout the report for uniformity. The subheadings and standard paragraph text is set in 12 pt Times New Roman, with 1.5-line spacing and justified text. In addition, the system feature requirements are given unique identifiers in the form REQ-SF [Feature Number]-[Requirement Number] (e.g., REQ-SF1-1). Unless otherwise stated, it is assumed that a higher-level priority assignment (High, Medium, Low) for a system feature is carried over to all sub-requirements.

2.1.3 Target Audience and Suggestions for Reading

This document is designed for various target groups:

- **Developers & Software Architects:** To facilitate coding, database design, API building, etc.
- **Quality Assurance (QA) Testers:** To create a complete set of test cases from the functional requirements.
- **Project Supervisors & FYP Evaluators:** To evaluate the technical scope and rigour of the project.
- **Domain Experts (Law Enforcement/Judiciary):** To ensure the system mirrors legal processes.

- Reading Suggestion: Section 1.3 External Interfaces and 1.4 System Features are important sections for the developer to pay attention to. Section 1.1 and 1.2 offer a high-level overview and evaluators and non-technical stakeholders should start there before moving on to the more general limitations.

2.1.4 Product Scope

NoiseSentinel is an automated system for digitizing the enforcement of rules and regulations pertaining to the noise pollution produced by the movement of vehicles, which is also IoT enabled. It aims to eliminate subjective and manual paper ticketing and introduce a data-driven workflow that is cryptographically secure. The system connects calibrated IoT noise sensors directly with a mobile app, providing irrefutable evidence when vehicles are over the 85.0 dBA legal noise limit. It categorises these offences into non-cognisable and cognisable offences, connecting the dots between the police on the beat and the courtroom and creating an unbroken chain of custody for electronic evidence from the street to the court.

2.2 Overall Description

2.2.1 Product Perspective

NoiseSentinel is a new stand-alone product to replace the disjointed and manual processes of noise pollution enforcement used today. An ecosystem that serves multiple platforms. The mobile subsystem serves as the data ingestion point through interaction with the external custom IoT hardware. All backend functions are independent, cloud-based, and include many complex business logic rules and Role Based Access Control (RBAC). The web subsystem is the management console for higher tier users. All operations are integrated with each other through RESTful API calls and a single SQL Server database.

2.2.2 Users, Classes and Characteristics

The platform has a very strict segregation of duties with the 5 user classes which are designed with specific access and interfaces according to the use environment.

User Class	Main Platform	Description
System Administrator	Web Portal	Manages user accounts, provisions IoT Device IDs and manages overall system health. Has the highest system privileges.
Police Officer	Mobile App	Works in the field to connect with IoT sensors, collect noise evidence and issue challans with a simplified UI.
Station Authority	Web Portal	Works from police stations to look at cognizable challans and convert them to First Information Reports (FIRs).
Court Authority	Web Portal	Functions as a Court Clerk, accepting FIRs, creating official Court Cases, and assigning to courts dockets.
Judge	Web Portal	Comprehends the information contained within the digital evidence which cannot be altered in a case and to make final Case Statements and legal verdicts.

Table 3: User classes with their respective responsibilities

2.2.3 Operating Environment

- **Mobile Application:** Optimized to work with Android 10.0+ and iOS 14.0+. Needs working Bluetooth, Wi-Fi and camera components.
- **Web Portal:** Supports any HTML5 compliant Web browser (Google Chrome, Mozilla Firefox, Safari, Microsoft Edge).
- **Backend Server:** This server is hosted on a Windows or Linux server that supports the .NET 8.0 runtime environment.
- **Database:** Microsoft SQL Server.
- **IoT Hardware:** Microcontroller with a calibrated sound level sensor module, such as ESP32 or Raspberry Pi.

2.2.4 Design and Implementation Constraints

- The backend can only be built using .NET 8.0 and has a 3-tier architecture (Technology Stack Constraint). Mobile app should be developed in React Native (Expo).
- All user authentication will be done using JSON Web Tokens (JWT). The system should be able to securely hash passwords with the ability to digitally sign the Emission Reports to render it tamper proof.
- **Connectivity Constraint:** Mobile devices on the street may have poor cellular networks and the application should temporarily store the information in the local mobile device and sync when the connection is restored.
- **Regulatory Constraint:** Strict legal threshold of 85.0 dBA for violation.

2.2.5 Assumptions and Dependencies

Assumptions:

- **Legal Admissibility:** Digitally signed, auto-generated environmental data (Emission Reports) are assumed to be recognized as a primary and legal document in court.
- **Hardware Availability & Endurance:** The physical IoT sensor modules (e.g., ESP32/Raspberry Pi with I2S microphones) are assumed to be available, weather resistant and able to maintain accurate calibration over long time periods of outdoor use.
- **User Compliance:** User of the field officers is expected to follow standard operating procedures and will set up the IoT device in a proper position in relation to the vehicle's exhaust before making an attempt to scan the vehicle.
- **Network Availability:** Offline capabilities are proposed but it is assumed that officers will eventually be able to connect to a cellular or Wi-Fi network, at the end of their shift, to synchronize the data they have with the central data.

Dependencies:

- **Third-Party Libraries:** The mobile app heavily utilizes Expo SDK, in particular expo-secure-store for secure, local storage, expo-camera for taking visual evidence, and react-native-ble-plx (or similar) for Bluetooth IoT communication.

- **Cloud Infrastructure:** The uptime of the system is directly dependent on the cloud hosting provider (e.g., Azure, AWS, or a local university server) that the .NET 8.0 Web API and the Microsoft SQL Server database is hosted on.
- **SMTP Service:** The multi-factor authentication system (OTP) requires a dependable external SMTP server (SendGrid or regular Gmail SMTP) to send verification emails within seconds.

2.3 External Interface Requirements

2.3.1 User Interfaces

- **Mobile Application (Police Officers):** Interface to be developed using React Native. The UI need to be a "Dark Mode" that can be used in outdoor settings, in order to reduce glare on the screen when exposed to sunlight. Navigation to be optimized for one-handed use, using bottom tab navigators. The scanning screen should show a real time decibel wave or gauge so that the IoT sensor data is immediately reflected in the screen.
- **Web Portal (Authorities & Judges):** Shall be modern in frontend development (e.g., React.js or Angular). It will make use of complex data grid elements for server-side pagination, sorting and advanced filtering. Different dashboards will be displayed as per the user's JWT role payload (for example: Pending cases will be shown in the Judge dashboard and the same will not be visible in the other dashboards and also the "Generate FIR" button will be hidden for the Judge dashboard).

2.3.2 Hardware Interfaces

- **IoT Sensor Communication:** The basic hardware interface is that of the mobile device and the acoustic sensor unit. The data characteristics of the hardware shall be subscribed to using Bluetooth Low Energy (BLE 4.2+) and the Generic Attribute Profile (GATT) in the mobile application. The hardware will send an array of decibel values (float) to the application at 10Hz while the scan is on when a 'Start' command is received.
- **Camera Integration:** The mobile app shall communicate with the smartphone's built-in camera. It is required that it uses auto-focus and auto-exposure APIs to make sure the license plate(s) captured from a vehicle is easy to read in different lighting conditions.

2.3.3 Software Interfaces

- **Database Interface:** The .NET 8.0 backend shall communicate with a relational database of Microsoft SQL Server. All interaction will be done via Entity Framework Core (EF Core 8) with all data accesses made using LINQ queries and EF Migrations for schema version control.
- **Native APIs of OS:** There is a need to use the native keystore of the mobile operating system (Android Keystore / iOS Secure Enclave) to store and encrypt the JWT access tokens so they cannot be extracted even in the event of a physical compromise.
- **File System Interface:** The back-end API should communicate with the file system of the server or a specific blob storage container to store and access the photographs that are uploaded and associated with the Challans.
- **Frontend Framework:** Frontend development and UI rendering is made using React.js (v19.2.0).

2.3.4 Communications Interfaces

- **RESTful Architecture:** All client-server communication must be RESTful (Representational State Transfer). The system will use standard HTTP verbs as follows: GET to retrieve records, POST to create new Challans/Users, PUT to set statuses and DELETE (soft delete only) for record invalidation.
- **Payload Formatting:** Data serialization between mobile app, Web Portal and Backend API shall be in JSON (JavaScript Object Notation) format only.
- **Security & Encryption:** All traffic on the network must be secure, encrypted over a secure connection (TLS 1.2 or later). All API requests must be made with an Authorization token with a valid Bearer [JWT].
- Future enhancements (optional) can be implemented such as email or SMS alerts/notifications of case changes.

2.4 System Features

2.4.1 Capture & Evidence Digitization from the IoT

2.4.1.1 Description and Priority

This feature with High priority includes the localised pairing of the mobile device with the hardware, the ingestion of acoustic data and the production of an unalterable digital record (the Emission Report) if the noise level is above the limit of 85.0 dBA.

2.4.1.2 Stimulus/Response Sequences

- User clicks on "Connect Device". System searches and pairs with the BLE MAC address of the authorized sensor.
- User clicks on "Start Scan". A trigger signal is sent to the hardware by App.
- 10 Seconds of Decibels Data are returned by Hardware.
- App algorithm analyses the array. When the average/peak is > 85.0 dBA, the App will ask the backend to create an Emission Report.
- Backend returns the Emission Report ID cryptographically signed.

2.4.1.3 Functional Requirements

REQ-SF1-1: Mobile application shall include the use of a BLE scanner to identify and filter out IoT devices in the vicinity of the mobile device with a particular UUID that is specific to the NoiseSentinel hardware.

REQ-SF1-2: Application shall compute moving average and peak values of the acoustic data array ingested to reduce the false positive occurrences due to sudden, unrelated background acoustics.

REQ-SF1-3: When threshold is crossed the system shall create an "Emission Report" object containing the data array, the time of crossing, the MAC address and the GPS coordinates.

REQ-SF1-4: The backend API shall produce the digital signature of the Emission Report using HMAC-SHA256 algorithm which ensures the immutability of the data.

2.4.2 Automated Challan Issuance

2.4.2.1 Description and Priority

The term used for the procedure in which the Police Officer secures the Emission Report to the offender's name and vehicle, and officially logs the legal violation. This process also has the High priority.

2.4.2.2 Stimulus/Response Sequences

- Emission Report screen is replaced by the Challan generation form.
- Officer enters the Vehicle Registration Number and Offender's CNIC.
- Officer takes a picture of the car.
- Officer completes the form.
- System categorizes the violation and saves the Challan, providing a confirmation receipt.

2.4.2.3 Functional Requirements

REQ-SF2-1: All the input fields shall be validated by using Regular Expressions (e.g. CNIC: XXXXX-XXXXXXX-X).

REQ-SF2-2: The mobile app should compress the image captured by the app to a max file size of 2MB, to optimize bandwidth usage, prior to sending the image to the server in a multipart/form-data request.

REQ-SF2-3: The background of the business logic shall consider the offender's history in the database. In case of repeat breach, or any decibel reading is beyond an extreme limit (as per the discretion of the department, for example > 100 dBA), the Challan shall be marked as "Cognizable".

REQ-SF2-4: The system shall support the storage of the Challan payload generated in the local queue in an encrypted manner in case the device is offline.

2.4.3 RBAC Legal Escalation Workflow

2.4.3.1 Description and Priority

The back-end process behind this, which sends the data to the right administrative or judicial platforms, allowing for a separation of powers. This process is also of high priority.

2.4.3.2 Stimulus/Response Sequences

- Station Authority will pick a "Cognizable Challan" which is pending and click the "Escalate to FIR" option.
- System associates the Challan with a new created FIR.

- The FIR is then submitted to "Court Authority" where a Court Docket Number and Judge ID are assigned, and thus a "Court Case" is born.
- Judge reads the Court Case and looks at the attached Emission Report and pictures and then writes a "Verdict".

2.4.3.3 Functional Requirements

REQ-SF3-1: The .NET Web API shall be using role-based attributes on all routing endpoints (e.g., [Authorize (Roles="Station Authority")]) to prevent the manipulation of URLs.

REQ-SF3-2: There should be a 1:1 relational link between escalating Challan and the FIR in the system.

REQ-SF3-3: The system should include a text editor interface to enable the Judge role to write their final Case Statement.

REQ-SF3-4: After submission of a Verdict, the entire hierarchy of Case/FIR/Challan shall be locked with the status as "Closed", and no changes shall be allowed in the database.

2.4.4 Secure Authentication & Auditing

2.4.4.1 Description and Priority

The centralized login system with Multi-Factor Authentication to make sure only verified personnel are logging in. Medium priority process.

2.4.4.2 Stimulus/Response Sequences

- User enters e-mail and password.
- System validates credentials and sends 6-digit OTP through SMTP.
- User types the OTP in the 5 minutes validity period.
- System validates OTP and generates a JWT.

2.4.4.3 Functional Requirements

REQ-SF4-1: The system shall use the BCrypt algorithm to hash user passwords with a work factor of 10 or more.

REQ-SF4-2: Backend API will generate a random, cryptographically secure string of 6 digits OTP and add it to the database with an expiration time of 5 minutes.

REQ-SF4-3: On successful verification the backend should provide a JWT with a user ID, Role and a normal expiration time (such as 8 hours).

2.5 Nonfunctional Requirements

2.5.1 Performance Requirements

The system shall be designed to perform quickly and respond to the user's requests under normal and peak use conditions, including:

- **Response Time:** 95% of expected API calls (not including large image uploads) should return a response to the client in 1.5 seconds.
- **Throughput:** The database should be able to support read and write access by a minimum of 50 concurrent field officers without deadlocking.
- **Data Retrieval:** Data grids on the dashboard should be paginated on the server side, with no more than 50 records per query to avoid bloating the browser memory and slowing down the loading times.

2.5.2 Safety Requirements

- **Operational Safety:** The mobile app shall present a removable warning message to Police Officers at start-up, to park their cars safely before trying to use the mobile app and/or connect to hardware.
- **Hardware Safety:** If officers are using the hardware for calibration, standard operating procedures should stipulate that hearing protection be worn when standard operating procedure requires the sensor to be exposed to the noise source for testing purposes, and the noise exceeds 100 dBA.

2.5.3 Security Requirements

- **Data at Rest:** Mobile device sensitive offline information (e.g. queued challans) shall be encrypted on the mobile device using the secure storage API provided by the mobile operating system using the AES-256 algorithm.

- **Data in Transit:** All data packets sent over the internet should be encrypted for Man-In-The-Middle (MITM) attacks using SSL/TLS 1.2+ certificates.

A JWT should never be stored in any format that can be easily accessed in the mobile app (for example, using regular AsyncStorage or localStorage). It is required to use secure keychains.

2.5.4 Usability Requirements

- The system will offer a simple and easy-to-use interface. Role based dashboards shall only show relevant features.
- **Accessibility:** Text colors and background contrasts on the mobile app should meet the Web Content Accessibility Guidelines (WCAG) AA standard, so that it can read in outdoor environments.
- **Efficiency of Use:** The mobile process to write noise tickets and create Challan should not need more than 5 screen taps to ensure officers are able to complete traffic stops in an efficient manner.

2.5.5 Reliability Requirements

- **Availability:** The back-end infrastructure shall be designed to meet the following target: 99.9% uptime for standard enforcement hours.
- **Fault Tolerance:** If the database connection is temporarily lost, the .NET backend should also automatically attempt to reinsert the data into the database after logging it to a temporary file system or cache. Consistency of data shall be ensured during all operations.

Recovery mechanisms shall recover system functionality following failures.

2.5.6 Maintainability / Supportability Requirements

- **Architecture:** The .NET 8.0 backend should follow a Clean Architecture approach, employing the Repository Pattern and Dependency Injection to achieve complete separation between the Data Access Layer and the Business Logic Layer.
- **Documentation:** The backend API shall be interactive, up-to-date API documentation for the front-end developers shall be auto-generated using Swagger (Swashbuckle).

2.5.7 Portability Requirements

- **Cross-Platform Compilation:** The React Native codebase should not be too specific of native code, and around 95% of the code should be common in both Android and iOS version.
- **Server Hosting:** The .NET 8.0 API should also support being containerized with Docker for deployment in Windows IIS, Linux, and cloud container instances.

2.5.8 Efficiency Requirements

- **Battery Consumption:** The mobile application should be responsible for the Battery Life Extension (BLE) connection state and after the 10-second scan, disconnect from the IoT sensor to preserve the smartphone's battery life.
- Background processes shall keep system load to a minimum.
- Optimization of database queries through the use of indexes and efficient design.

2.6 Domain Requirements

- **Legal Data Compliance:** The system should provide compliance to the specific regional Environmental Protection Agency (EPA) standards or Traffic Code thresholds. In this project, the business logic is defined as an explicit hard-coded number, which is 85.0 dBA for vehicular noise pollution.
- **Digital Evidence Admissibility:** The architecture of the system ensures that each generated piece of evidence (Emission Report) is timestamped with an irreversible time of creation, location of origin and has a cryptographic hash to prove that the evidence has not been tampered with since its creation. The system shall also keep a well-structured database of the challans, cases, FIRs, hearings as well as user data.

Records shall be searchable and retrievable according to the filters like date, location, violator details and case status.

The system shall provide report generation capabilities for legal, administrative and performance analysis reasons.

Ensure that the system respects data privacy and security regulations for handling sensitive user and case information.

Chapter 3
Use Case Analysis

3.1 Introduction

The chapter is dedicated to the Use Case Analysis of the NoiseSentinel platform, which represents the functional requirements from the end-users' point of view. It covers the relationship between the actors of the system (Police Officers, Station Authorities, Court Authorities, Judges, Administrators, and Public), and software components of the platform, in the mobile application, the web portal, and the backend API. Case models and detailed tabular specification are used to specify the boundaries of the system, the main 1x1 paths of execution and the alternative paths of execution, including all enforcement and judicial workflows

3.2 Use Case Model

The Use Case Model is used to specify the functional requirements of the system from a user's point of view. It is structured representation of the way in which the important features of the system interact with the external actors so that they can accomplish certain operational and legal objectives. The model makes the interactions visible, allowing to define clear system borders and to ensure separation of duties, necessary for a legally traceable enforcement workflow, for all platform parts.

3.2.1 System Actors

The platform consists of seven different actors, who have different permissions and operational contexts.

- **Police Officer (Primary Actor):** Field personnel operating the mobile application. Responsible for pairing with IoT device, performing noise and emission tests, identification of vehicles and accused persons and issuance of digital challans with photographs.
- **Station Authority (Primary Actor):** Administration staff using the web portal from police stations. Checks challans raised by officers, accepts/rejects the challans based on the quality of the evidence and escalates cognizable offences to formal FIRs.
- **Court Authority (Primary Actor):** Court administrative staff using the web portal. Receives escalated FIRs, formalisation of court cases, scheduling of court cases, and assigning of judges to court cases.

- **Judge (Primary Actor):** Judicial personnel who use the web portal. Evaluates the entire evidence trail associated with a case assigned to it and enters a legally binding verdict to resolve the case.
- **System Administrator (Primary Actor):** System management personnel using web portal with highest privileges. Performs tasks related to user account management in all roles, enrols IoT devices, and controls the system configuration.
- **Public / Accused (Primary Actor):** Members of public, usually the accused persons identified on challans, who are able to access the public facing module of the web portal without having to be authenticated. Can verify case status using Vehicle no and CNIC after OTP and can confirm the payment of pending challans.
- **ESP32 IoT Device (Secondary Actor):** The physical device that field officers carry. Can communicate via Bluetooth Low Energy with the mobile application, performs noise and emission tests on command, and returns structured JSON data that contains sensor readings and violation classifications.

3.2.2 Primary Use Cases

The following are the main functions available in the NoiseSentinel ecosystem:

- **Register and Manage Users:** Administrator creates and manages user accounts for all roles and register IoT devices in the system.
- **Authenticate User:** All authenticated roles log into their respective interfaces with email and password and have a JWT based session management.
- **Pair IoT Device and Conduct Test:** Police Officer pairs the mobile application with the ESP32 device via BLE and perform tests to generate the noise or emissions and get structured results.
- **Issue Digital Challan:** Police Officer issues a formal digital challan with IoT sensor information, manual violations and vehicle information, accused information and photographic evidence.
- **Review and Escalate Challan:** Station Authority receives the challans sent by them for review, approves or rejects it and escalates approved cognizable challans to FIRs.

- **Manage Court Case:** Court Authority receives escalated FIRs, forms court cases, set court hearings and assigns judges.
- **Record Verdict:** Judge reads through all of the evidence of an assigned case and records a verdict, thereby closing the case.
- **Public Case Lookup and Payment:** Public user verifies his/her identity through OTP and looks for the case status and challan information and confirms the payment of unpaid challans.

3.3 Use Cases Description

Use Case 01: Authenticate User

Field	Description
Use Case Name	User Login & Authentication
Description	Users who are registered log in to the mobile app or web portal with their email and password. If the credentials are validated, then a JWT will be issued that gives access to the role specific features.
Primary Actor	All roles that have been verified: Police Officer, Station Authority, Court Authority, Judge, Administrator.
Secondary Actor	None
Precondition	The user account has to be created by the Administrator and the email address has to be verified before it is allowed to log in.
Dependency	None
Generalization	None

Table 4: Use case 01 (User Login & Authentication)

Basic Flow:

1. User accesses the mobile app or goes to the web portal login page.
2. The user enters his/her registered email address and password.
3. User clicks Login.
4. The system checks the credentials with the database.
5. When validated successfully, the backend returns a signed JWT with user's ID, user's role and expiry time.

6. The client saves the JWT in encrypted storage and sends the user to his/her role-specific dashboard.

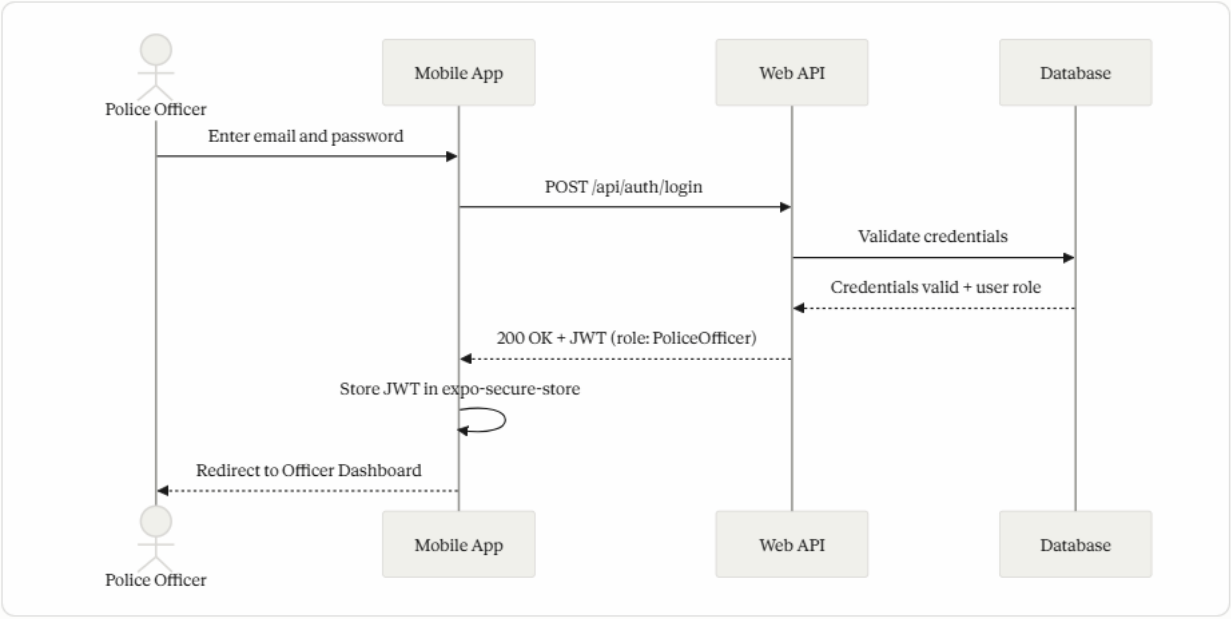
Post condition: User is logged in and has access to all features he/she can use according to his/her role.

Alternative Flow:

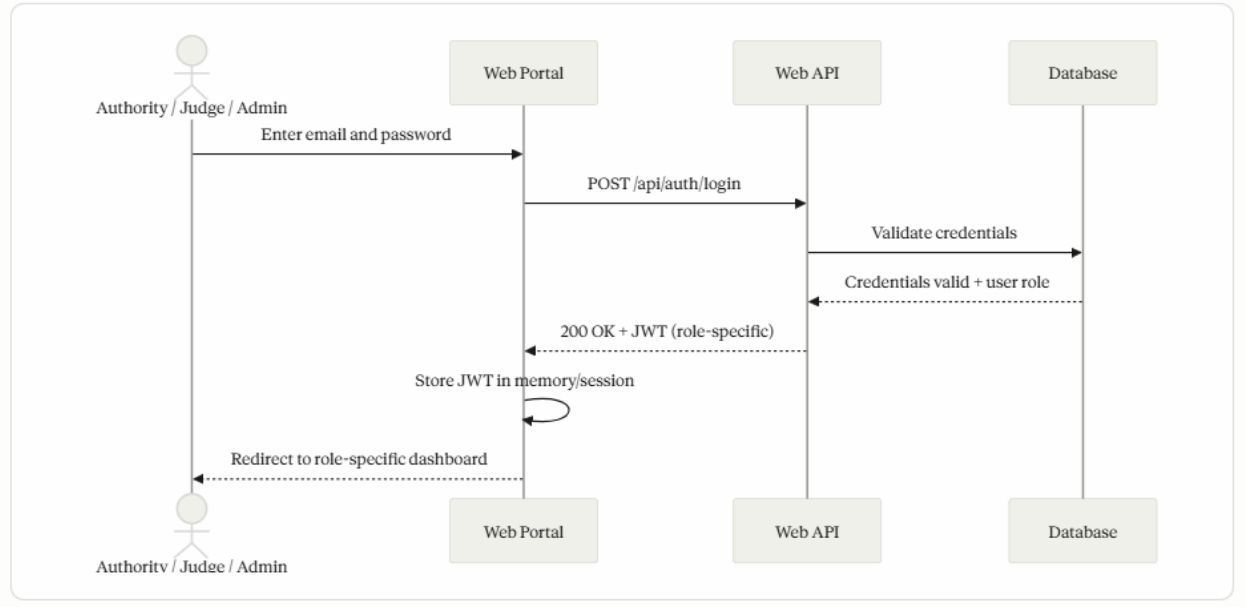
1. User logs in using incorrect email and/or password.
2. The credentials are verified, and a mismatch is detected.
3. An error response is returned, and a plain language error message is displayed that indicates invalid credentials.
4. User stays on the logon screen.

Post condition: No access. No token is issued. The user may retry.

Login Flow — Mobile Application (Police Officer)



Login Flow — Web Portal (Station Authority, Court Authority, Judge, Admin)



Use Case 02: Pair IoT Device and Conduct Test

Field	Description
Use Case Name	Pair IoT Device and Conduct Test
Description	The Police Officer connects the mobile app to the ESP32 IoT device via Bluetooth Low Energy and starts a noise test or an emission test. The device performs the test and provides a structured result including the sensor readings and a classification for the violation.
Primary Actor	Police Officer
Secondary Actor	ESP32 IoT Device
Precondition	Officer is logged in on the mobile application. The ESP32 is powered on and running advertisement on BLE. The device LED is red, indicating it is awaiting a connection.
Dependency	Implicitly includes "Connect to IoT via Bluetooth".
Generalization	None

Table 5: Use case 02 (Pair IoT Device and Conduct Test)

Basic Flow

Following is the flow of **Noise Test**:

1. Officer goes to the IoT pairing screen and selects Connect Device.

2. The app searches for BLE devices that are compatible with the NoiseSentinel service UUID and shows the discovered device.
3. Officer chooses the device. BLE is connected and the device LED turns to yellow.
4. Officer taps Start Noise Test.
5. The application sets the device's control characteristic to a value of START_NOISE_TEST.
6. The device takes 10 seconds of audio captures, derives the dBA reading based on RMS calculation, classifies it with the set noise thresholds and gives back a single JSON.
7. The result is sent to the application and the dBA reading and violation classification is shown to the officer.
8. The officer then moves on to creating the challan, with the sensor data already entered.

Following is the flow of **Emission Test:**

1. Repeat steps 1–4 above except officer presses Start Emission Test.
2. The application writes a START_EMISSION_TEST command.
3. The device enters thirty seconds warming phase. Every three seconds, the application gets countdown progress updates from the device.
4. Device goes to sampling phase for 10 seconds and obtains gas sensor data.
5. The final JSON output with CO level, HC level, NOx level, CO2 level, and violation classification is sent by the device.
6. The application shows the readings, and the officer goes for the challan creation.

Post condition: Test result will be saved in the application and can be linked with a new challan.

Alternative Flow

Applies if the **Device Not Found**

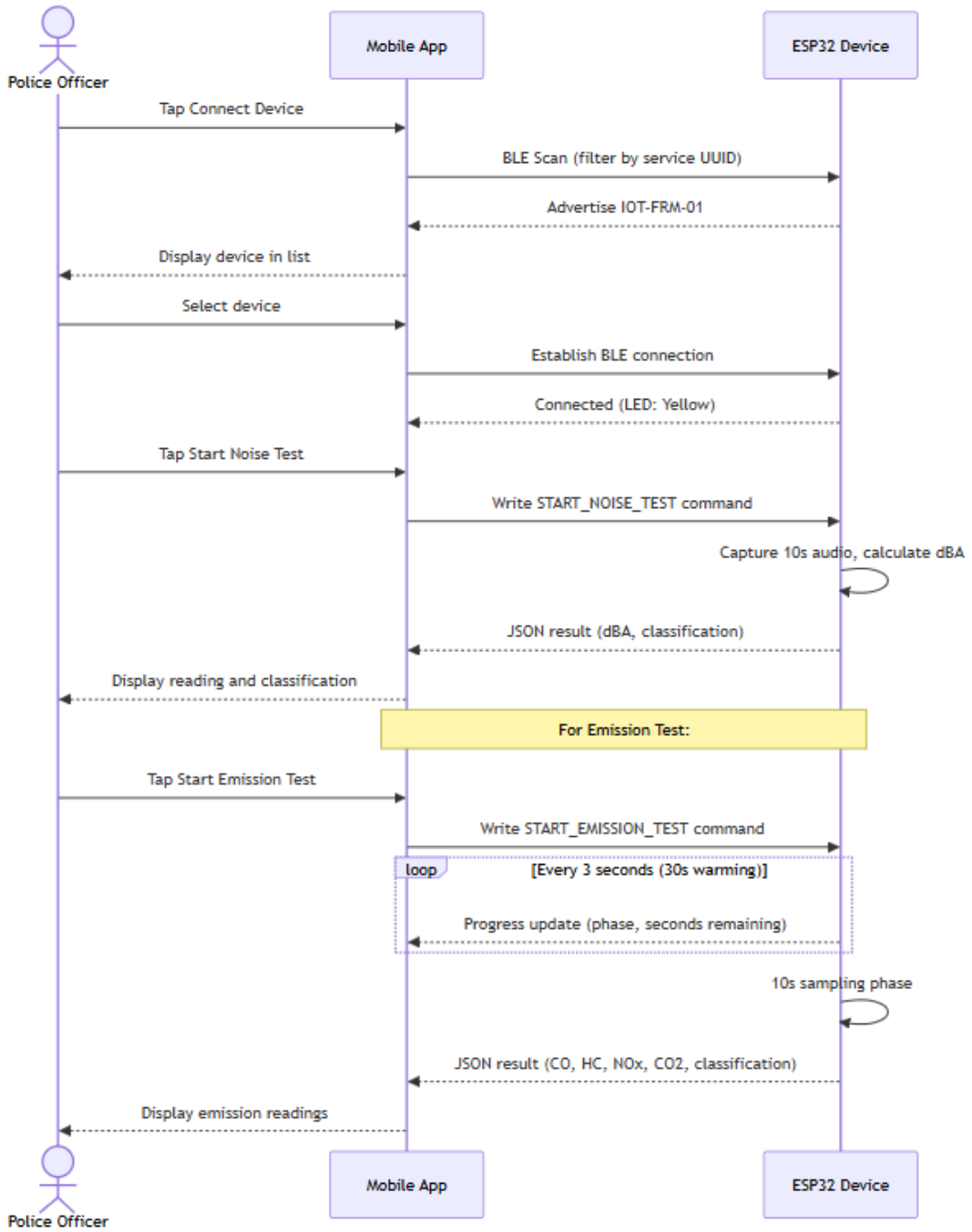
1. Officer taps Connect Device.
2. When scanning BLE, the scan is not completed, and no device is found with the NoiseSentinel UUID.
3. Displays Device not found and asks officer to make sure device is on and in range.

Post condition: There is no connection. Officer can repeat the scan.

Applies if **BLE Disconnection During Test**

1. In the mid of a test, the BLE connection is lost.
2. The application senses when the connection is lost, ends the test state, and logs a Connection lost message.
3. It resets the test state when it disconnects.

Post Condition: Nothing is stored. Officer will need to reconnect and rerun the test.



Use Case 03: Issue Digital Challan

Field	Description
Use Case Name	Issue Digital Challan
Description	The Police Officer prepares a formal on-line challan against the vehicle and accused person with IoT sensor data (in case of non-traffic violations) or manual data (in case of traffic violations) and photographic evidence. The challan is sent to the backend and recorded as a legal document.
Primary Actor	Police Officer
Secondary Actor	None
Precondition	The officer has logged in into mobile application. For non-traffic challans, an IoT test result is available from Use Case 02.
Dependency	For Non-Traffic Challans: extends Use Case 02 (Pair IoT Device and Conduct Test)
Generalization	None

Table 6: Use case 03 (Issue Digital Challan)

Basic Flow

Following is the flow for **Non-Traffic Challan (IoT-evidenced):**

1. Officer clicks on Create Non-Traffic Challan and chooses the violation category (Noise/Emission).
2. Officer completes the IoT test as per Use Case 02. The Challan form is auto filled with sensor data.
3. Officer looks for the vehicle in the police database by plate number and chooses the corresponding record or fills in new information regarding the vehicle.
4. Officer finds accused through CNIC and clicks on the correct one or clicks on NEW ACCUSED DETAILS if the accused is not found.
5. Officer takes some pictures of the vehicle as evidence.
6. Bank account information is shown to the accused for reference (read-only and it is auto populated from the backend configuration).

7. Officer checks the entire form and clicks on Submit Challan.
8. A confirmation dialog is displayed. Officer confirms.
9. The application POSTs the challan to the backend API /api/challan.
10. Backend generates challan and associates the emission report to it and gives a success response along with Challan ID.
11. The officer will see a success message on the application.

Following is the flow that follows for **Traffic Challan (Manual):**

1. Officer clicks Create Traffic Challan and chooses the appropriate violation type from the list.
2. Officer targets and chooses the car and suspect.
3. Officer takes a picture of the car.
4. Officer completes and submits the form.
5. Backend generates challan record and gives confirmation.

Post condition: A new challan record in Unpaid status is added in the database and is visible in officer's challan history and in station authority dashboard.

Alternative Flow - Vehicle or Accused Not Found:

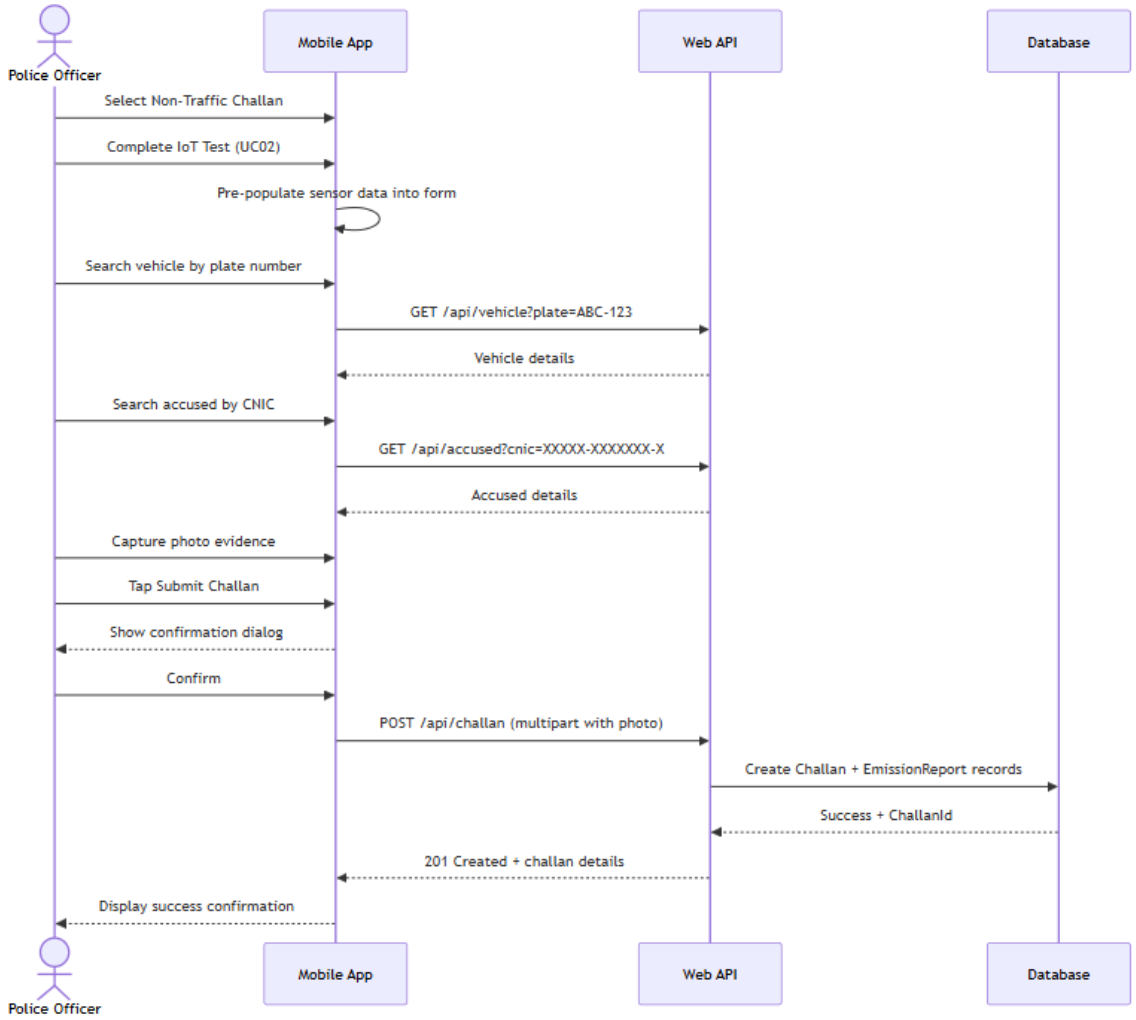
1. Officer tries and searches for the vehicle according to the plate number. No matching record is found.
2. Officer clicks on Add New Vehicle and types the vehicle information.
3. New vehicle record is added to the system and associated with the challan.

Post-condition: Challan is created with the newly created vehicle record.

Form Validation Failure:

1. Submit Challan is tapped by Officer with one or more field in Challan being left blank or improperly formatted.
2. Application marks invalid fields and shows validation messages.
3. Only when all fields are valid, the submission is allowed

Post condition: There is no API call. Officer edits and re-submits



Use Case 04: Review & Escalate Challan

Field	Description
Use Case Name	Review and Escalate Challan
Description	The Station Authority reviews the challans submitted through the web portal by the field officers, reviews the evidence and either approves the challan for further processing or rejects the challan with a recorded reason. Approved Cognizable Challans can be elevated to FIRs
Primary Actor	Station Authority
Secondary Actor	None

Precondition	One or more challans in Unpaid status are present in the system. The Station Authority is signed in on the Web Portal.
Dependency	Cognizable escalation extends Use Case 05 (Generate FIR)
Generalization	None

Table 7: Use case 04 (Review and Escalate Challan)

Basic Flow – (Approve and Escalate)

Following is the flow:

1. Station Authority logs into Web Portal and goes to Dashboard – Pending Challans.
2. Authority selects a challan and clicks Review.
3. Full details of the challan are shown in the system — violation type, sensor readings, emission report, photographic proof, vehicle details and accused details.
4. The evidence is then looked at by Authority and Authority Approves.
5. In case of cognizable offenses, Authority clicks Escalate to FIR. The system moves on to Use Case 05.

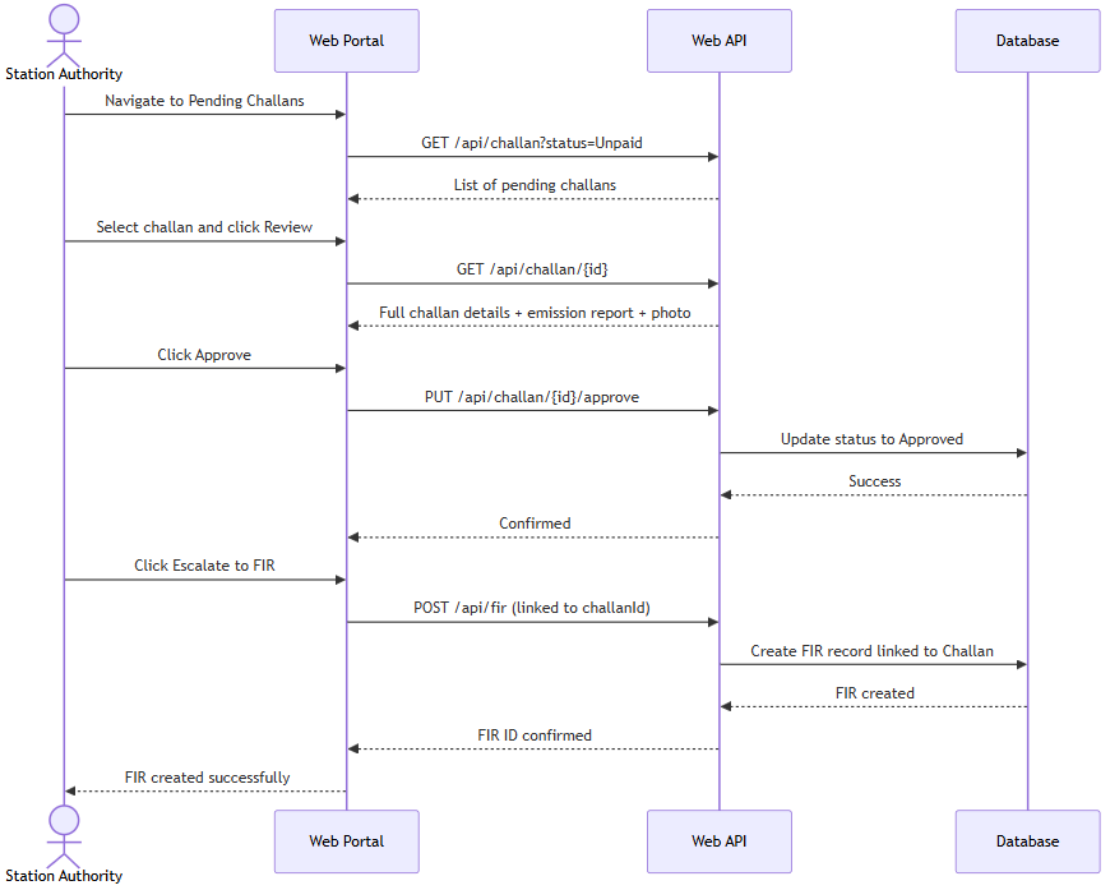
Post condition: Status of Challan is updated to Approved. An escalated FIR is created, and the challan status is changed to Escalated if escalated.

Alternative Flow

Applies in case of **Reject Challan:**

1. Challan is reviewed by the Authority, and it is found that evidence is inadequate, or the violation has not been duly noted.
2. The Authority clicks Reject and types in a reason for the rejection.
3. The rejection reason is captured, and the status of the challan is changed to Rejected and it aborts the legal workflow for that challan.

Post condition: The challan is marked Rejected. It is still present in the system for auditing purposes but cannot be escalated.



Use Case 05: Generate First Information Report (FIR)

Field	Description
Use Case Name	Generate First Information Report (FIR)
Description	The Station Authority converts formally an approved cognisable challan into a FIR, and makes a linked FIR record in the system which is seen by the Court Authority for further processing.
Primary Actor	Station Authority
Secondary Actor	None

Field	Description
Precondition	Challan has to be in Approved status and it has to be noted as Cognizable Offence. The Station Authority is signed in to the web portal.
Dependency	Extends Use Case 04 (Review and Escalate Challan)
Generalization	None

Table 8: Use case 05 (Generate First Information Report - FIR)

Basic Flow:

1. After the challan is approved as a part of Use Case 04, Station Authority clicks Escalate to FIR.
2. The system asks for any further comments from administration.
3. If applicable, Authority makes remarks and confirms escalation.
4. It creates a new FIR entity in the database, which is foreign keyed to the originating challan.
5. The challan status is changed to Escalated.
6. The FIR is displayed on the Court Authority dashboard.

Post-condition: An FIR is generated and connected to the challan. The Court Authority now has the ability to view and take action on FIR.

Use Case 06: Manage Court Case

Field	Description
Use Case Name	Manage Court Case
Description	The Court Authority gets elevated FIRs, formalizes cases, schedules hearings and assigns judges to adjudicate cases.
Primary Actor	Court Authority
Secondary Actor	None

Field	Description
Precondition	An FIR is in the system with an Escalated status. The Court Authority is connected with web portal.
Dependency	Extends Use Case 05 (Generate FIR)
Generalization	None

Table 9: Use case 06 (Manage Court Cases)

Basic Flow:

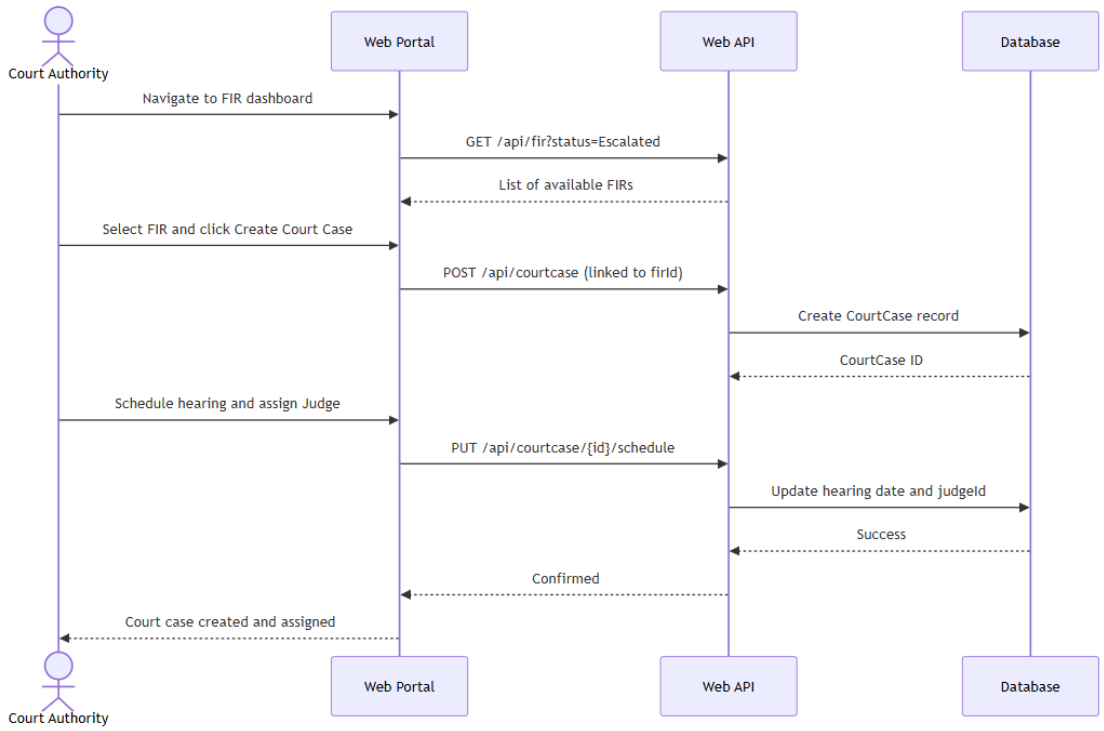
1. Court Authority logs into the web portal and goes to the FIRs dashboard available.
2. Authority chooses an FIR and checks the attached challan information and evidence.
3. Authority clicks Create Court Case.
4. System generates court case entity related to the FIR, challan and emission report.
5. Authority sets a date for the hearing and chooses a judge currently available in the system from the list of registered judges.
6. The system determines, and records, the judge and the date of the hearing.
7. The judge assigned to see the case is now able to see the case on his/her dashboard.

Post-condition: A court case is created, and it is added a judge and the judge can see the full evidence chain.

Alternative Flow -Reschedule Hearing:

1. After a hearing has been assigned, a scheduling conflict is identified by Court Authority.
2. Case is selected by Authority and Reschedule Hearing is clicked.
3. Authority types in new hearing date, confirms.
4. The system updates the hearing record.

Post-condition: Hearing date is posted and available to the judge.



Use Case 07: Record Verdict

Field	Description
Use Case Name	Record Verdict
Description	The whole digital evidence chain of a court case including the original challan, emission report and photographic evidence is submitted to the assigned Judge who issues a formal verdict and case statement. An entire case hierarchy is closed once it is submitted.
Primary Actor	Judge
Secondary Actor	None
Precondition	A court case was assigned to the Judge and a hearing was scheduled. The Judge has access to the web portal.
Dependency	Extends Use Case 06 (Manage Court Case)

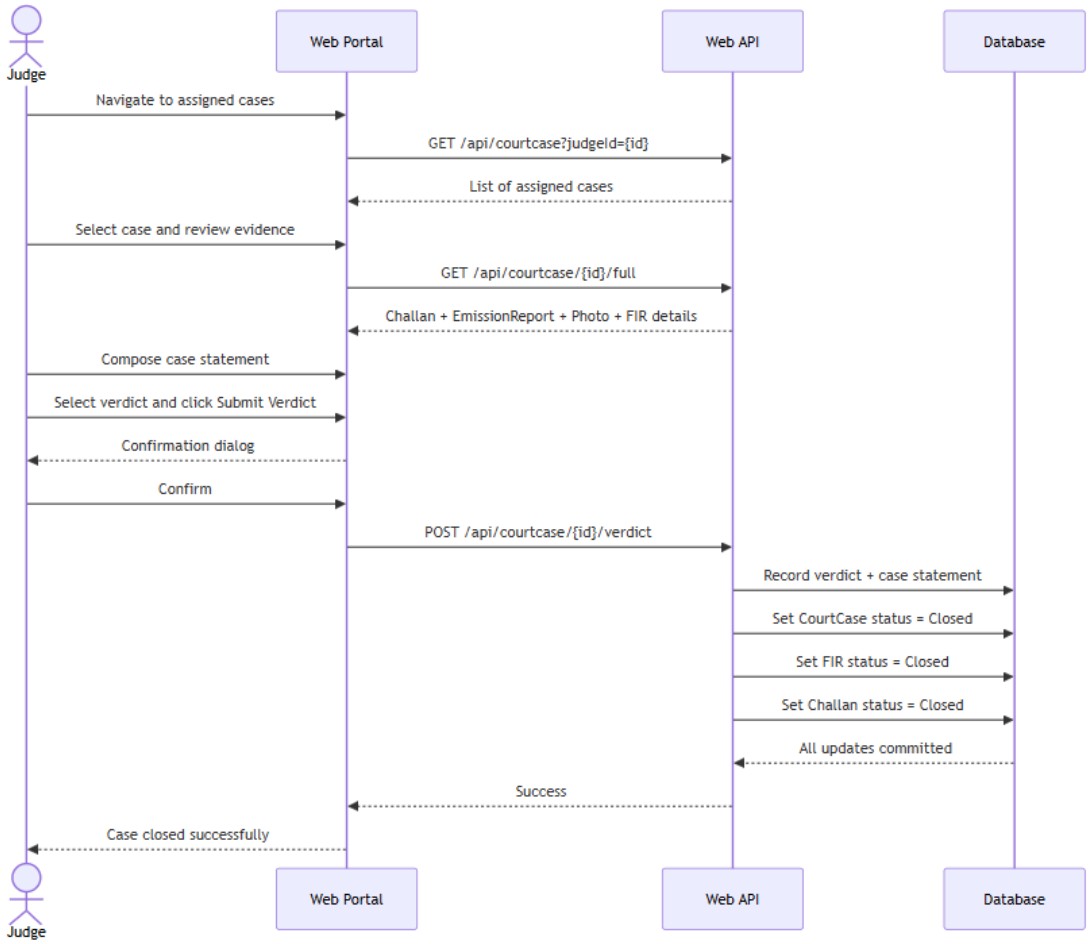
Field	Description
Generalization	None

Table 10: Use case 07 (Record Verdict)

Basic Flow:

1. Judge logs into web portal and sees his/her assigned cases in the Judge dashboard.
2. Judge reads through details of the challan, IoT emission report, sensor data, the type of violation and photos.
3. Judge has the hearing inside the system and writes his/her formal case statement in the system's text processor.
4. Judge clicks Submit Verdict to choose a verdict outcome.
5. A confirmation dialog is displayed. Judge confirms.

Post-condition: Case, FIR and Challan are all closed. Verdict is final and cannot be changed.



Use Case 08: Public Case Lookup and Payment

Field	Description
Use Case Name	Public Case Lookup and Payment
Description	An accused person or member of public accesses public facing module of web portal, without requiring account. They check their identity through OTP, check the status of their case and the corresponding challans, and ensure that no challan remains unpaid by manually transferring the money from the bank.
Primary Actor	Public / Accused
Secondary Actor	SMTP2GO Email Service
Precondition	There is at least one challan in the system for the queried vehicle number and CNIC number. The accused can have access to the email address provided at the time of searching.
Dependency	None
Generalization	None

Table 11: Use case 08 (Public Case Lookup and Payment)

Basic Flow:

1. Accused logs onto the public case status page on noisesentinel.tech. No log in is required.
2. Accused provides registration of his/her vehicle, CNIC and email address.
3. SMTP2GO delivers the 6-digit OTP to the entered e-mail address.
4. Accused fetches the OTP from his/her email and types it on the verification screen.
5. OTP (valid for 15 minutes) is verified and all the challans for the vehicle and CNIC are displayed.
6. Accused reviews information about challan type of violation, fine, station from which issued, when issued, when due and current status.

7. If the challan is unpaid, accused clicks Pay Now.
8. Payment dialog appears with Bank Name, Branch Code, IBAN and Account Number (full bank account details).
9. Accused makes the bank transfer on his/her own banking channel.
10. Accused is brought back to the portal and clicks on Confirm Payment.
11. The status of the challan changes to Paid and the same is shown in the display.

Post-condition: The challan gets marked as Paid in the database. This change is reflected right away on the public portal as well as on the dashboards of officers and authorities.

Alternative Flow - OTP Expired:

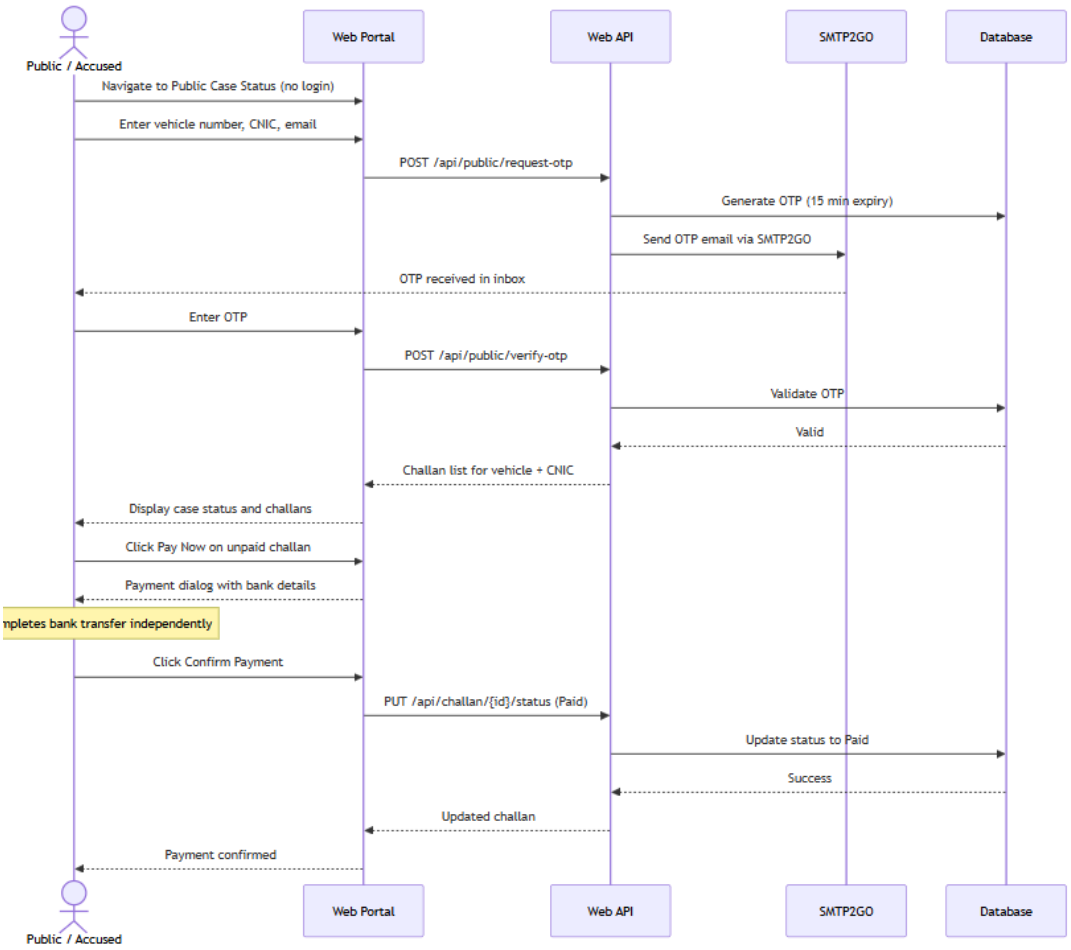
1. There is more than 15 minutes gap between the accused entering the OTP and the OTP has expired.
2. OTP expired message is displayed in the system.
3. Accused clicks Resend Code.
4. The system generates a new OTP and sends to the same e-mail address.

Post-condition**:** New OTP gets issued. The accused can try to verify again.

Alternative Flow - No Records Found:

1. Accused enters his/her vehicle number and CNIC. Once the OTP is verified, no matching challans can be found.
2. The no records found message is displayed in the system.

Post-condition: Data is not shown. This session does not do any changes to the database.



Use Case 09: Register and Manage Users and IoT Devices

Field	Description
Use Case Name	Register and Manage Users and IoT Devices
Description	The System Administrator sets up and maintains user accounts for all roles in the platform and registers IoT devices to pair with user accounts in the field.
Primary Actor	System Administrator
Secondary Actor	SMTP2GO Email Service
Precondition	Administrator has logged in to the web portal.

Field	Description
Dependency	None
Generalization	None

Table 12: Use case 09 (Register and Manage Users and IoT Devices)

Basic Flow - Create User Account:

1. Administrator goes to User Management on the admin dashboard.
2. Administrator clicks Add User and enters the new user's name, email, badge/staff number, user's station, and role.
3. Administrator submits the form.
4. Creating user and sending an email verification OTP to the new user.
5. OTP is sent to the new customer, and he/she goes to the verification screen and activates his/her account.
6. User can now login with his credentials.

3.4 Use Case Relationships Summary

Below is a table summarising the relationships between the use cases in the NoiseSentinel system.

Relationship	Use Cases Involved	Description
Extends	UC03 extends UC02	A Non-Traffic Challan is only issued following the completion of an IoT test.
Extends	UC05 extends UC04	FIR can only be created after review and approval of the Challan.
Extends	UC06 extends UC05	A Court Case can only be created after an FIR has been generated.

Table 13: Use case Relationships

Chapter 4:
System Design

Introduction:

The NoiseSentinel platform complete system design is detailed in this chapter, taking the software requirements defined in Chapter 2 and turning them into a concrete architectural blueprint. It includes information on layered system architecture, domain model, database schema, data dictionary, and various UML behavioural diagrams such as class diagram, sequence diagram, activity diagram, state transition diagram, component diagram, deployment diagram, and swimlane diagram. All diagrams are shown as the system was actually built and deployed at noisesentinel.tech.

4.1 Architecture Diagram

The architectural design of the NoiseSentinel system is based on a 3-Tier Layered Architecture to separate concerns, increase scalability and maintainability. Role-specific UI/UX for the Presentation Layer exists in the form of a React Native mobile application for the field officers and a web portal for the higher authorities.

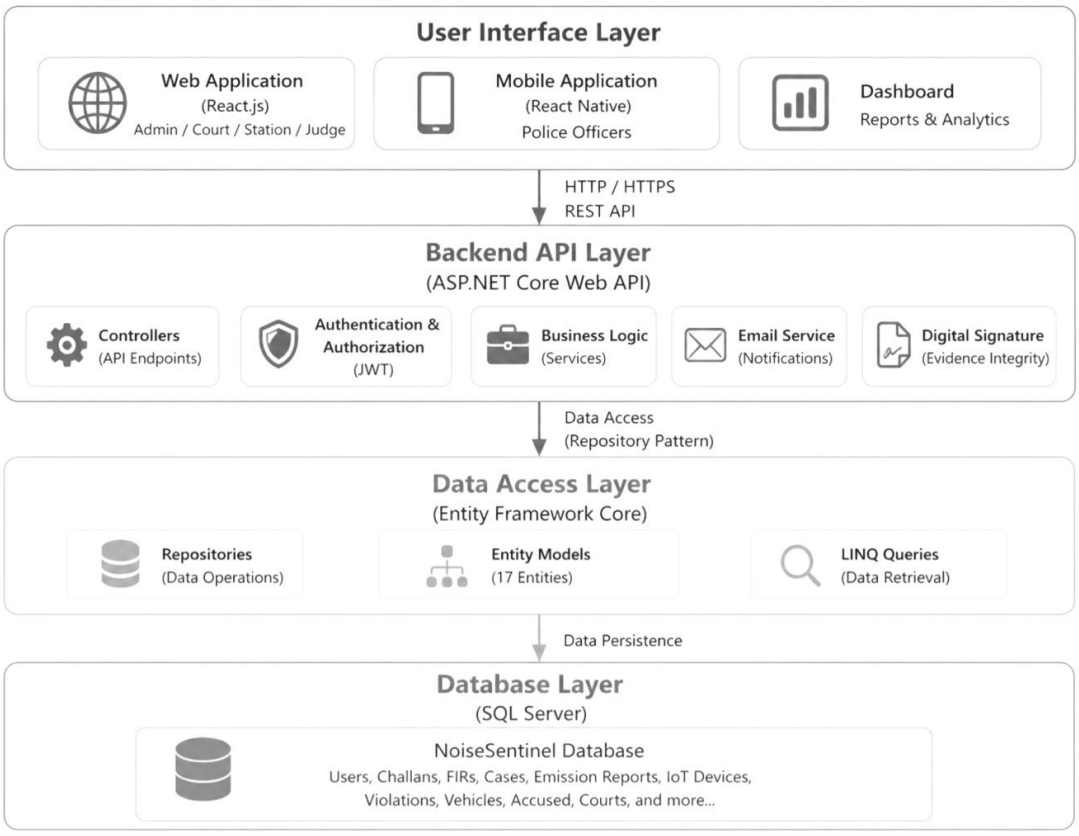


Figure 3: Architecture diagram for NoiseSentinel

4.2 Domain Model

The Domain Model represents the concepts in the NoiseSentinel problem space and the relationships between them. The key entities of this domain are System User (differentiated by their roles), IoT Sensor, Emission Report, Challan, FIR, and Court Case. Relationships ensure that a Police Officer operates an IoT Sensor, from which an Emission Report is created. This report is conceptually tied to a Challan that can later be raised to an FIR and eventually considered as a Court Case. This model connects the dots between the law enforcement vernacular and software objects.

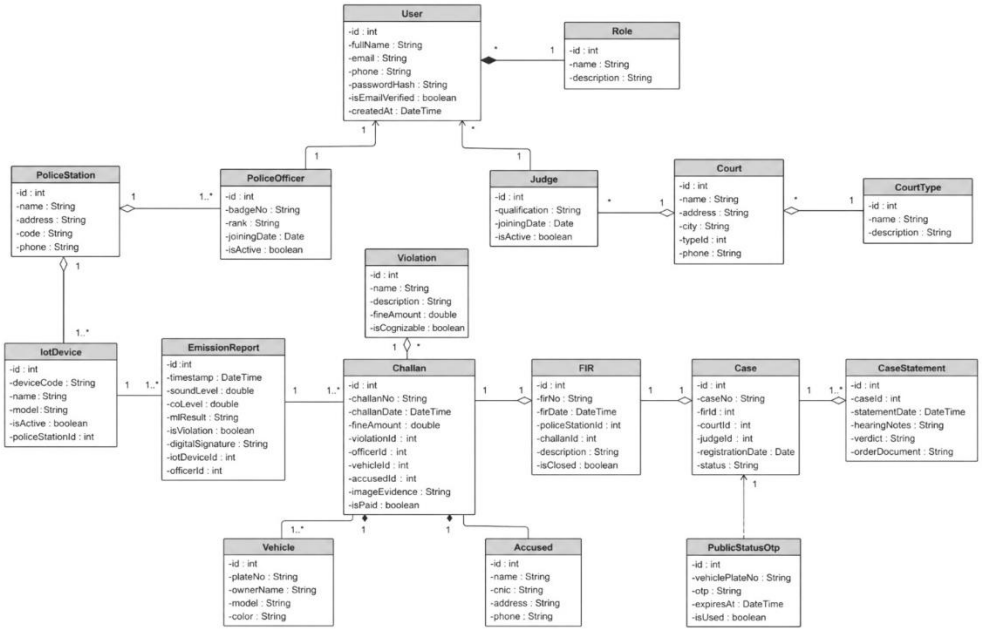


Figure 4: Domain model for NoiseSentinel

4.3 Entity Relationship Diagram with Data Dictionary

The logical schema of the centralized database is derived from the Entity Relationship Diagram (ERD). It shows primary keys, foreign keys, and multiplicity among tables to enforce Data Normalization.

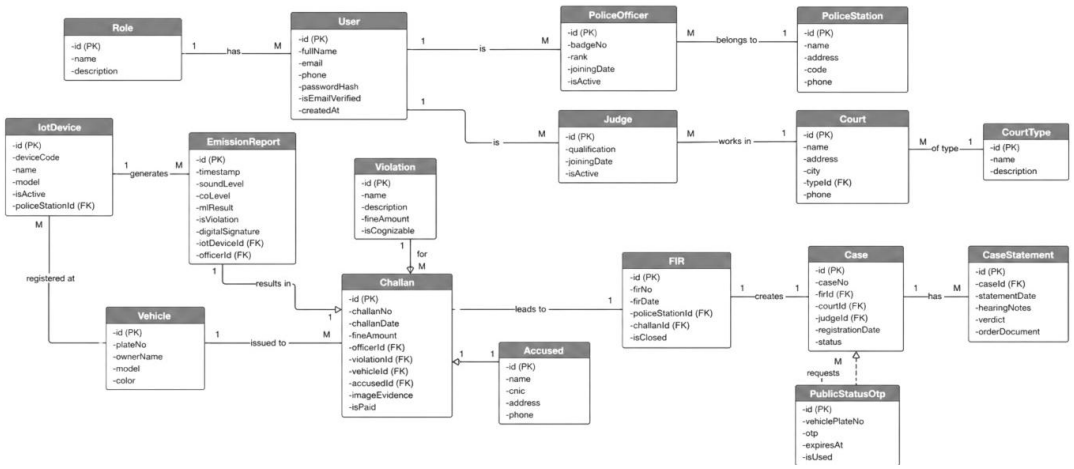


Figure 5: Entity Relationship Diagram for NoiseSentinel

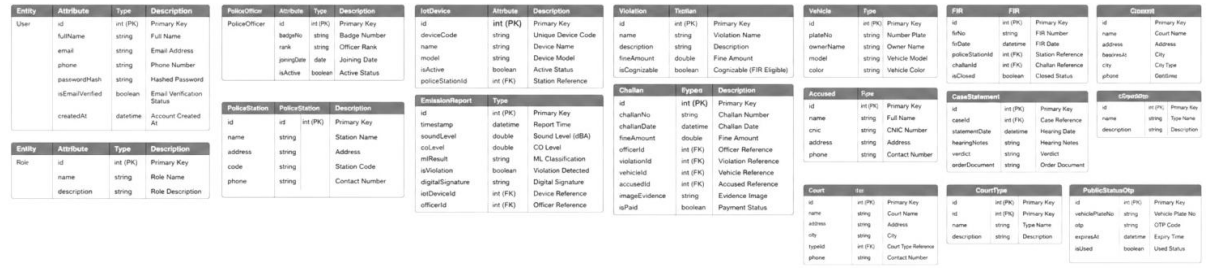


Figure 6: Data dictionary for ERD of NoiseSentinel

4.4 Class Diagram

The Class Diagram is a static representation of the object-oriented design of the system, showing the attributes, methods, access modifiers of each class in the .NET backend. It is used for defining concrete classes for controllers, services and models. For instance, the ChallanService class has a method named CreateChallan(), and the AuthService has a method called GenerateOTP() and another one called VerifyJWT(). Repository interfaces are also implemented by concrete classes of data access, as depicted in the diagram, and this is done with adherence to Dependency Injection principles.

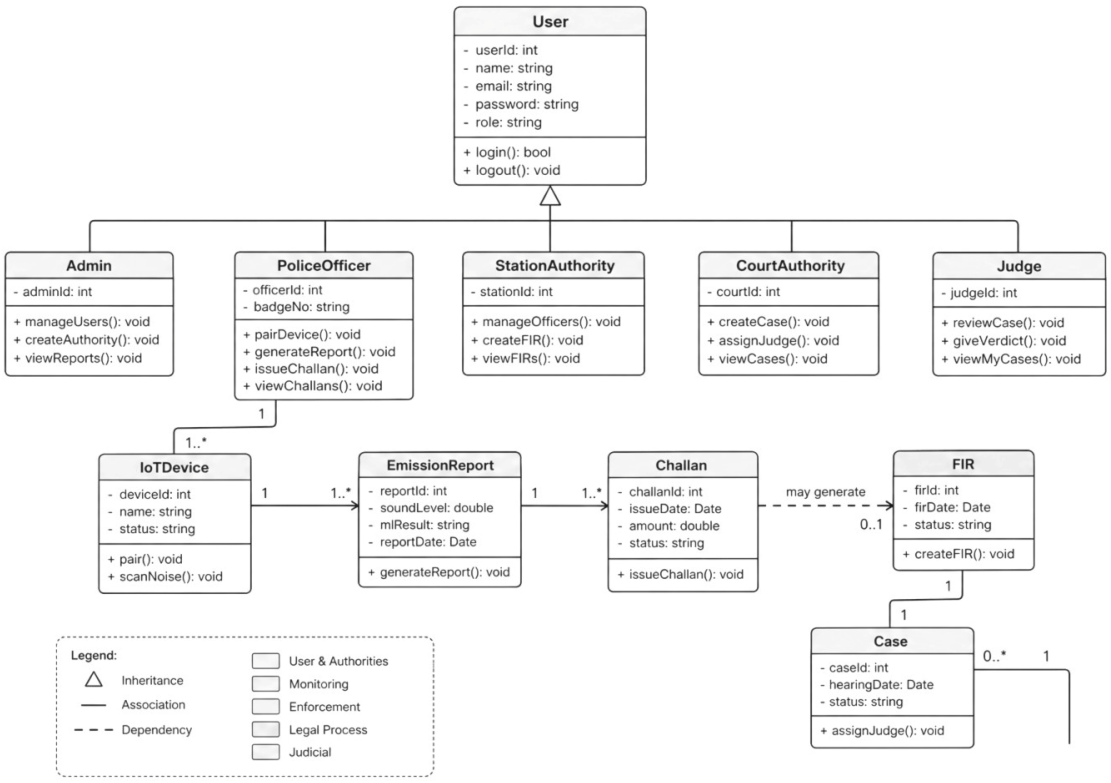


Figure 7: Class Diagram for NoiseSentinel

4.5 Sequence Diagram

The Sequence Diagram reflects the message sequence (dynamically) among objects in a time sequence to accomplish a use case. For example, the "Capture Emission Report" sequence starts with a scan trigger from the Mobile UI to the IoT Controller. The hardware will deliver an array of decibels back to the Mobile Client which will then make a POST request to the backend API. API calls Evidence Service to verify threshold, hashes payload, stores it in the database through Repository layer and returns success response and Report ID back to mobile application.

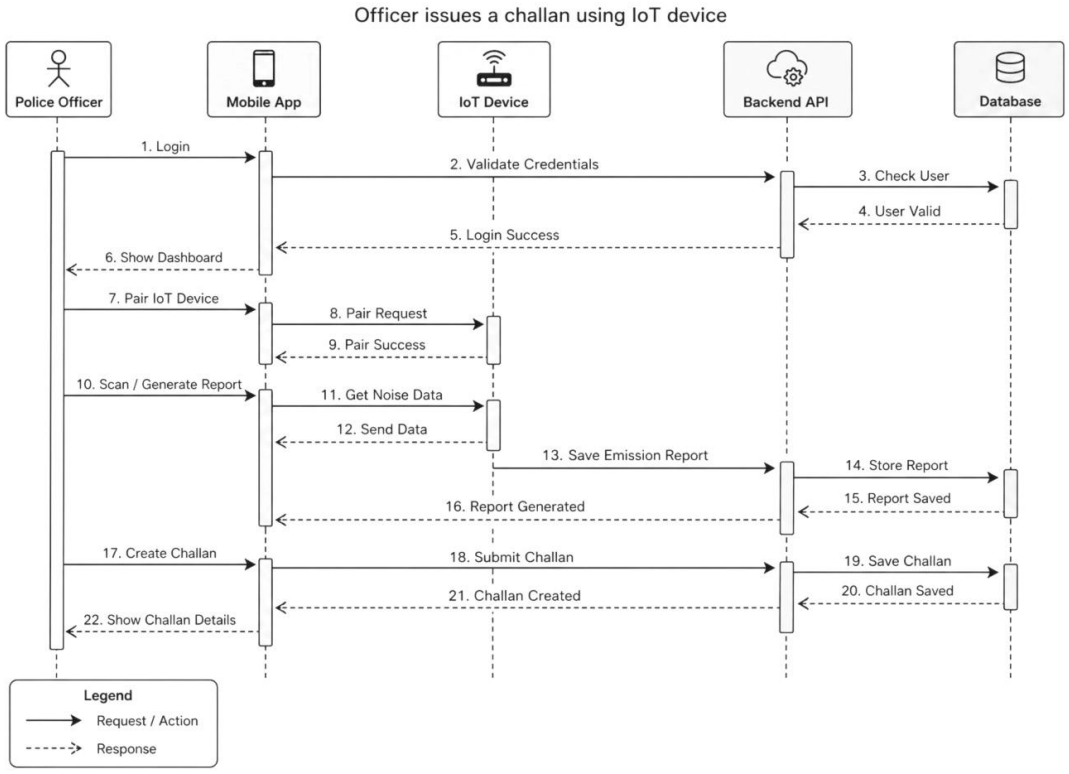


Figure 8: Sequence diagram for NoiseSentinel

4.6 Activity Diagram

The Activity Diagram shows the procedural flow of control and the decision-making logic in the NoiseSentinel workflows. It shows synchronization points, branching and parallel activities. One of the key activity flows is the legal escalation, which begins with the issuing of a Challan, is followed by the decision diamond "Is Cognizable?" and branches. If it is not cognizable, the flow is terminated with a normal fine output, but if it is, then the flow goes to the Station Authority swimlane for the generation of FIR and then to the Court Authority and Judge swimlanes, thereby depicting the smooth process of handing over.

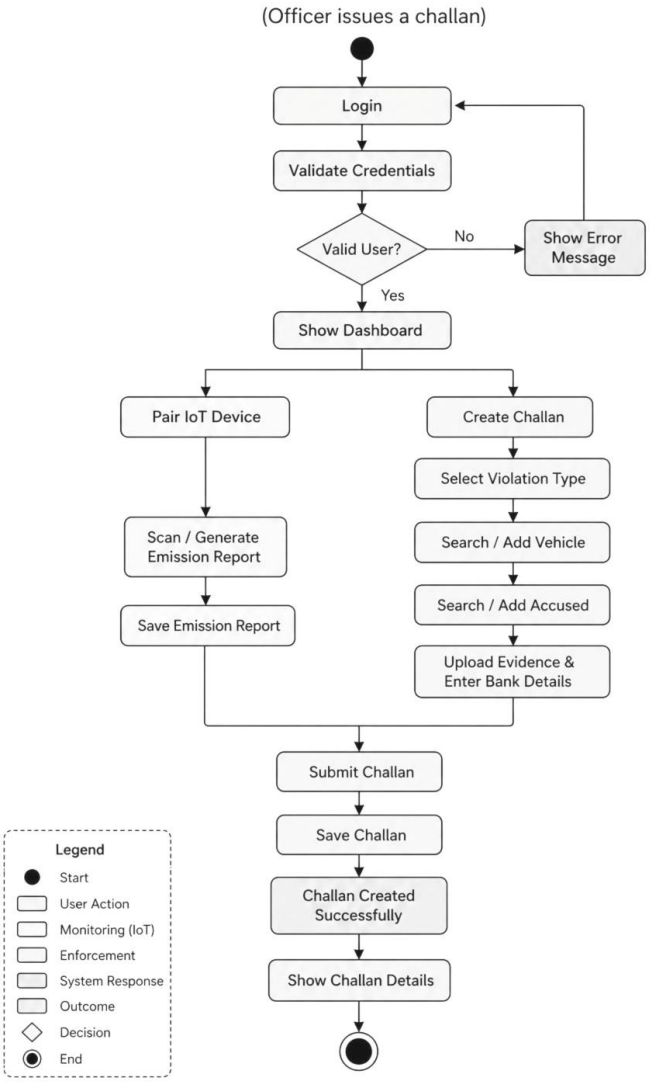


Figure 9: Activity diagram for NoiseSentinel

4.8 Component Diagram

The Component Diagram is used to show the physical structure of the software modules and how they are related. It demonstrates the React Native Client Component (compiled as a mobile package) interacting with the cloud hosted .NET Web Application Component through REST API interfaces. It also outlines the internal parts of the back end, including the Security Module (generates JWT), the IoT Integration Interface and the Database Access Module (Entity Framework Core), making the necessary and provided interfaces between the different modular blocks explicit.

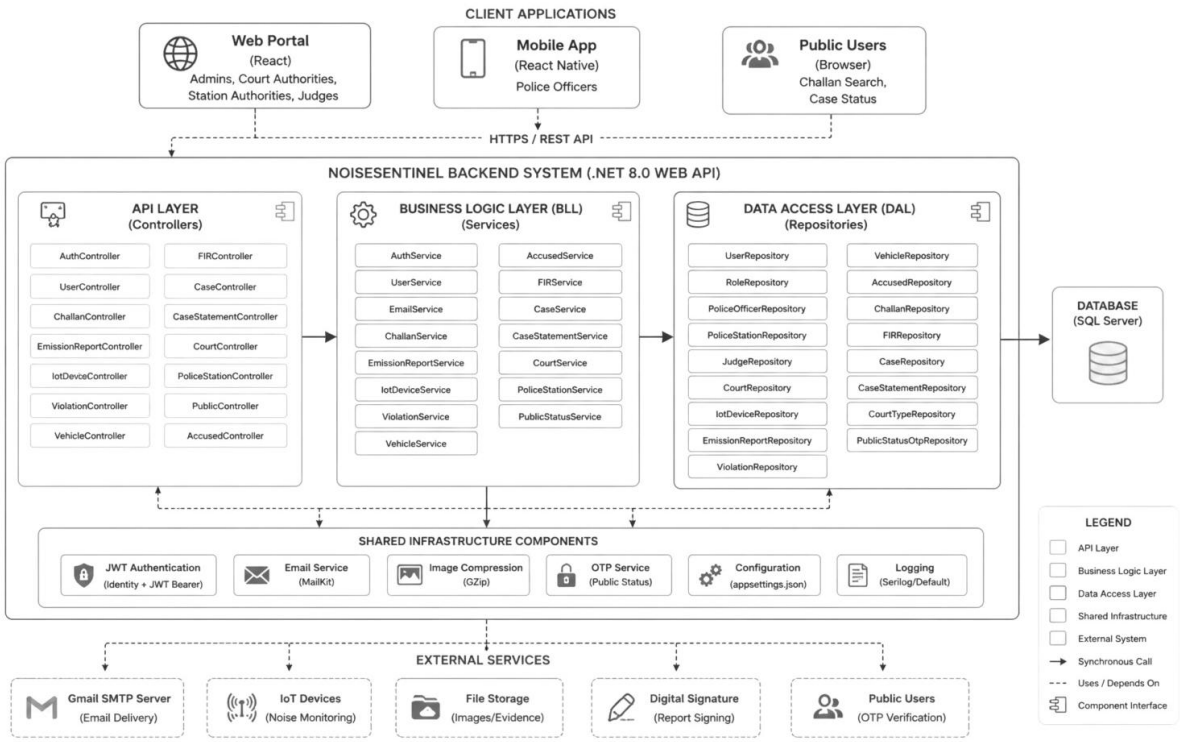


Figure 10: Component diagram for NoiseSentinel

4.9 Deployment Diagram

The Deployment Diagram shows how the software components relate to the physical hardware infrastructure. It represents the different nodes of the NoiseSentinel ecosystem: the Police Officer's Smartphone node (which hosts the mobile app and which communicates with the IoT Microcontroller node via Bluetooth), the Cloud Server node (which hosts the web server and the .NET application), and the Database Server node (hosting the MS SQL Server). Encrypted HTTPS/TLS for web traffic and TCP/IP for database queries are clearly annotated on association lines that connect these operational nodes.

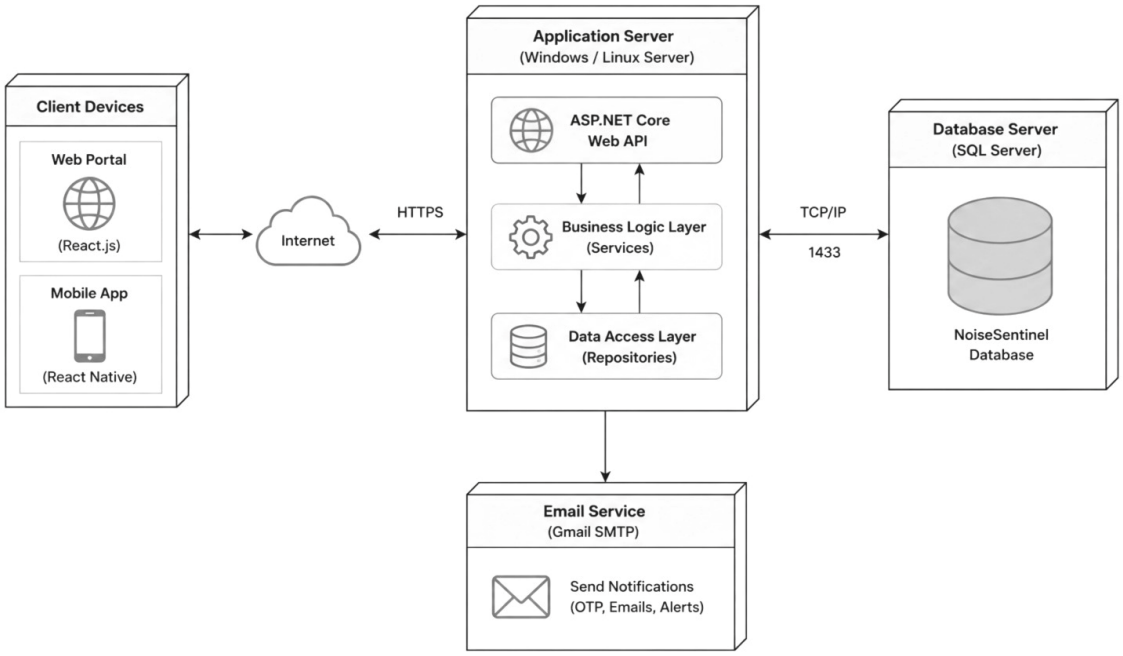


Figure 11: Deployment diagram for NoiseSentinel

4.10 Swimlane Activity Diagram

The Swimlane Activity Diagram (Cross-Functional Flowchart) captures the dynamic process and control flow of the NoiseSentinel platform's primary process of issuing a digital challan and escalating the same to a formal FIR. This diagram is organized vertically into lanes to identify specific system actors and technical components that explicitly show where responsibility is handed off. It captures the entire workflow of an action taken by a field officer which gets easily passed on through the mobile app and is securely processed and verified by the backend API and database and is ultimately shared with the station authority through the web portal, thus maintaining a transparent and unbroken digital evidence chain.

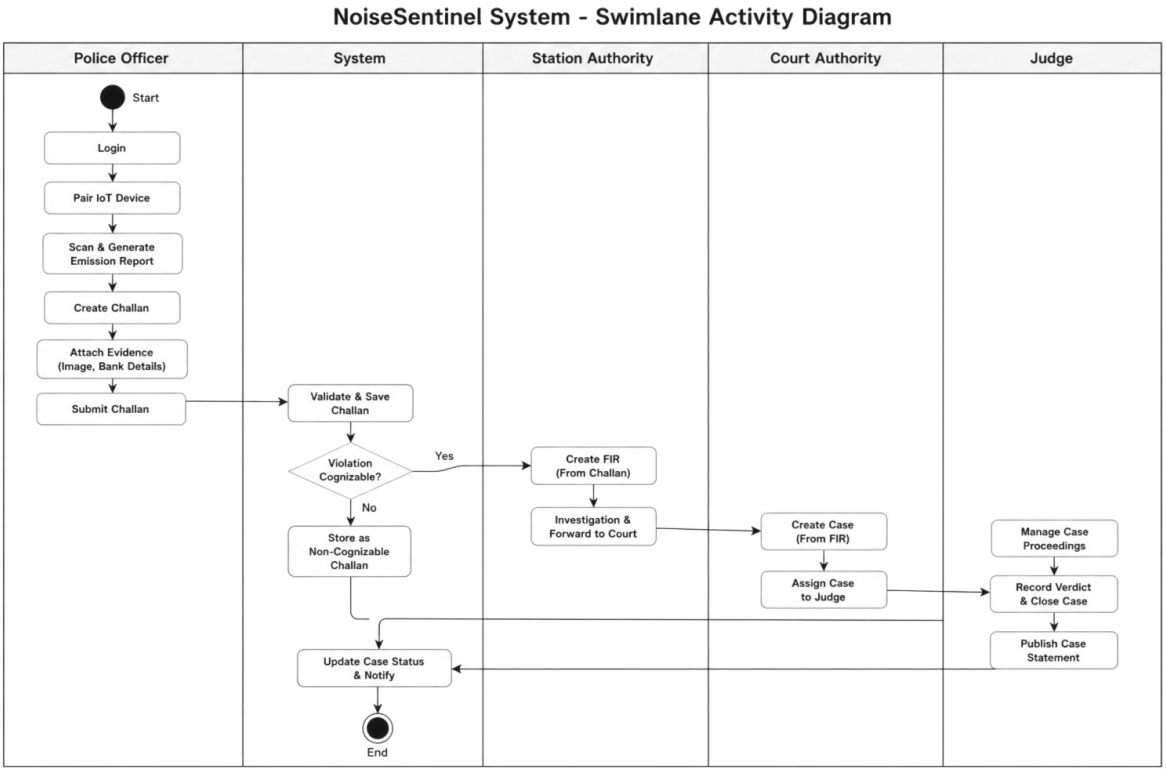


Figure 12: Swimlane activity diagram for NoiseSentinel

Chapter 5:

Implementation

Introduction

This chapter presents how the NoiseSentinel platform was built on the ground and how the architectural design and software requirements specified in previous chapters were translated into a deployed system. It includes pseudocode for the main algorithmic operations, full technology stack for all four subsystems, production deployment environment, development tools employed, and coding standards and version control employed during the project. All of the content in this chapter is based on the way the system was actually constructed and deployed at noisesentinel.tech.

5.1 Core Algorithmic Workflows

NoiseSentinel platform was designed with a series of well-connected workflows, that together constitute the entire enforcement and judicial pipeline. The actual logic in the system is implemented as pseudocode algorithms, one for each workflow.

5.1.1 Workflow for User Registration and Email Verification

New user accounts are created by the Administrator through the web portal. When it is created, the system sends an OTP to the email address of the new user for confirmation before the account is active. Previous to displaying case information, the public case lookup module uses this workflow to verify the accused's identity.

```
    START ALGORITHM RegisterAndVerifyUser
        Name, Email, Password, Role, BadgeNumber, Station is all
required.
        (Invoked by POST /api/auth/register by the Administrator)

        STEP 1: Validate all input fields
            If ANY required field is left blank OR email format is
wrong,
                Return a 400 Bad Request containing validation errors.
            END IF


        STEP 2: See if there are any duplicate emails.
```

5.1.2 IoT Device Test Execution Workflow

This is the workflow for the BLE communication between the mobile application and the ESP32 for noise and emission tests. The device performs the test automatically and provides the detailed structured JSON result. There is no streaming of raw data: the mobile app sends a command and expects a result.

5.1.3 Digital Challan Creation Workflow

As soon as the test result comes, the officer fills up a challan form with the details of the vehicle and accused and submits it. The backend generates the emission report and challan as related documents and automatically fills bank information from a centralised config.

```
START ALGORITHM CreateChallan
    INPUT: ChallanType, ViolationType, TestResult (may be null),
           VehiclePlate, AccusedCNIC, PhotoFile, OfficerJWT

    STEP 1: Validate officer's identity
        Claims = JWT.Validate(OfficerJWT, SecretKey)
        IF Claims.Role != "PoliceOfficer" THEN
            RETURN 403 Forbidden
        END IF

END ALGORITHM
```

5.2 Components, Libraries and Web Services

5.2.1 Libraries and Frameworks

ESP32 Firmware

Library	Version	Purpose
Arduino ESP32 Board Support	Latest stable	ESP32 hardware abstraction and BLE Arduino library
ArduinoJson	6.21.0+	JSON serialisation and deserialisation for BLE protocol

Mobile Application (React Native)

Library	Version	Purpose
React Native	0.72.10	Cross-platform mobile framework
Expo SDK	49	Dev tools and native modules
Android	4.14.2	Mobile app development for Android and iOS devices
Axios	1.6.2	HTTP client for REST API communication
react-native-ble-plx	3.1.2	Bluetooth Low Energy device scanning and communication
date-fns	2.30.0	Date formatting and manipulation

Table 14: Mobile Application libraries used for NoiseSentinel

Web API (ASP.NET Core)

Library	Version	Purpose
ASP.NET Core	.NET 8	Web API framework and HTTP pipeline
Entity Framework Core	8.x	ORM for database access and migrations
Microsoft.EntityFrameworkCore.SqlServer	8.x	EF Core provider for Azure SQL Server

Table 15: Web API libraries used for NoiseSentinel

Web Portal (React)

Library	Version	Purpose
React	18.x	UI component framework
Vite	5.x	Build tool and dev server

Library	Version	Purpose
Axios	1.x	HTTP client for REST API communication
Material UI (MUI)	5.x	Role specific dashboard components library
React Router	6.x	Client-side routing and route guards

Table 16: Web portal libraries used for NoiseSentinel

5.2.2 API Endpoints

The key REST API endpoints provided by the Web API backend are summarised in the following table.

Method	Endpoint	Auth Required	Role	Description
GET	/api/countries	No	Anyone	Returns a list of all countries
POST	/api/auth/register	Yes	Admin	Create new user account
GET	/api/challan	Yes	Officer, SA	Get challan list
POST	/api/challan	Yes	Officer	Add new challan
PUT	/api/challan/{id}/status	No	Public	Update challan payment status
GET	/api/emissionreport/{id}	Yes	Officer, SA, CA, Judge	Get emission report
POST	/api/fir	Yes	SA, CA	Create FIR
POST	/api/accused	Yes	Officer	Add new accused person

Method	Endpoint	Auth Required	Role	Description
POST	/api/public/request-otp	No	Public	Request OTP for case lookup
POST	/api/public/verify-otp	No	Public	Verify OTP and get case data
POST	/api/health	No	Any	API health check

Table 17: API endpoints for NoiseSentinel

5.3 Deployment Environment

The NoiseSentinel platform operates in a production environment on noisesentinel.tech with the following infrastructure pieces.

5.3.1 Production Infrastructure

- **Compute Node:** DigitalOcean droplet with Ubuntu Linux. The following tools are installed on the droplet: Docker Engine and Docker Compose. The Web API, Web Portal and Caddy reverse proxy are all deployed as separate Docker containers and orchestrated by one docker-compose.yml file in the root of the application repository.

The Compose file contains the definition of three services:

- **webapi** — builds based on the source code located in src/NoiseSentinel.WebApi/Dockerfile. All environment variables (connection string, JWT secret, SMTP credentials) are provided to the .NET Core application through the Compose environment configuration, using a .env file which is not version controlled.
- **webportal** — builds made from src/Noisesentinel.WebPortal/Dockerfile, generates a static build of Vite and uses Nginx on internal port 80 to serve it. VITE_API_BASE_URL build argument is being injected at build time.
- **Cloudflare DNS:** noisesentinel.tech is hosted by Cloudflare. The DigitalOcean droplet IP is used in DNS A records. Cloudflare is in Full Strict SSL mode which is an extra

layer of security between the droplet and the public internet. At the Cloudflare edge, DDoS protection and caching are provided.

5.3.2 Firewall Configuration

The following UFW rules are applied to the droplet to control inbound traffic:

Port	Protocol	Purpose
22	TCP	SSH access for administration
80	TCP	HTTP (redirected to HTTPS by Caddy)
443	TCP	HTTPS traffic

All other ports are blocked. The Web API (port 5200) and portal (internal port 80) are never directly exposed to the internet and are only accessible inside the Docker network.

5.3.3 Mobile Application Build

Expo Application Services (EAS) are used to build the React Native mobile app. src/utils/constants.ts contains the production API base URL, which is set to https://noisesentinel.tech/api. For Android, release builds are created in the form of APKs and are provided for dissemination to field officers. iOS deployments are marked as a future target and not part of the current production release.

5.4 Tools and Technologies

5.4.1 Development Tools

Tool	Version	Purpose
Visual Studio 2022	17.x	Main IDE for ASP.NET Core Web API — building, testing, and managing EF Core migrations
Visual Studio Code	1.8x	IDE for React Native mobile app and React web portal development

Tool	Version	Purpose
Arduino IDE	1.8.19+	Firmware development, compilation and upload to ESP32 hardware
Git	2.x	Distributed version control
GitHub	—	Remote repository hosting, branch management, and pull request reviews

Table 18: Development tools used for NoiseSentinel

5.4.2 Technology Stack Summary

Technology	Version	Subsystem	Purpose
ESP32 Arduino Framework	—	IoT Firmware	Microcontroller programming environment
React Native	0.72.10	Mobile App	Cross-platform Android application framework
react-native-ble-plx	3.1.2	Mobile App	Bluetooth Low Energy communication
ASP.NET Core	.NET 8	Web API	Back-end REST API framework
Entity Framework Core	8.x	Web API	ORM and database migrations
Azure SQL Database	—	Database	Relational database for production
React	18.x	Web Portal	UI component framework

Table 19: Development Technologies used for NoiseSentinel

5.5 Coding Standards and Best Practices

5.5.1 Naming Conventions

All four subsystems consistently used naming conventions to make things easier to read and to reduce cognitive load during coding and code review.

Context	Convention	Example
C# Classes and Interfaces	PascalCase	ChallanService, IUserRepository
C# Methods	PascalCase	CreateChallan(), GenerateOTP()
C# Private Fields	camelCase with underscore prefix	_bankSettings, _challanRepository
C# Configuration Keys	PascalCase	BankDetails, JwtSettings

Table 20: Naming conventions used in coding

5.5.2 Architectural Patterns

- **Clean Architecture (Backend):** All the responsibilities of the Web API backend are strictly separated into three layers. The DAL only handles access to the data via EF Core repositories. All business logic is in service classes that are accessed via interfaces in the BLL. In the Web API layer, all you have are controllers that validate inputs, call services and return responses. Controllers do not contain any business logic and there are no direct database calls in the Web API layer.
- **Repository Pattern:** All database operations are placed inside repository classes which implement specified interfaces. Services are not based on concrete implementations, but on interfaces that are passed to the service via the constructor. This way, the data access implementation can be altered or mocked without affecting the business logic.
- **Dependency Injection:** All services, repositories and configuration objects are registered in Program.cs and injected via their constructors throughout the application. No use of a static class or service locator pattern. This contributes to testability and dependency graphs that are easy to understand.

5.5.3 Security Practices

- Before being stored, passwords are securely hashed using BCrypt with a work factor of 12. The plain text password is never logged or stored.
- JWTs are signed using HMAC-SHA256 with an environment variable provided secret key. The tokens expire in 24 hours
- The JWT is validated and at the middleware level the role claim of the JWT is checked on every request sent to every authenticated endpoint before the controller action is executed.

5.5.4 Code Organisation

All subsystems use a uniform internal directory structure that allows easy access to all files of a particular feature across a single directory without having to go through several unrelated ones to get to the files of interest.

5.6 Version Control

The NoiseSentinel project is version controlled using Git, with GitHub as remote repository host. All four subsystems (esp32-firmware, mobile-app, Web API and web-portal) are kept in a single monorepo so changes in any of the subsystems can be tracked and reviewed with others.

5.6.1 Branching Strategy

Branch	Purpose
main	Production-ready code. Only merged into after review. Mirror of noisesentinel.tech deployment.
feature/*	Individual feature branches created for each new capability, e.g. feature/ble-emission-test, feature/public-payment.
fix/*	Fix branches created to resolve a specific bug, e.g. fix/otp-expiry-check.
deploy/*	Branches for deployment configuration changes.

Table 21: Branching strategies for version control

5.6.2 Commit Practices

Commit messages are formatted in a particular way to provide a clear indication of the purpose and scope of the change:

```
[subsystem] short description of change
```

Examples:

```
[api] add UpdateChallanStatusAsync to ChallanService
[mobile] implement BLE emission test progress notifications
[firmware] add MQ-7 sensor warm-up state machine
[portal] add public case lookup OTP verification screen
[infra] configure Caddy reverse proxy for production domain
[db] add EF Core migration for EmissionReports table
```

5.6.3 Version Tagging

Git tags are used to indicate important milestone releases, as a stable point of reference throughout the project.

Tag	Description
v0.2.0	Additions to the project scaffold for all subsystems
v0.5.0	BLE connect between mobile app and ESP32 completed
v0.8.0	End to end full challan and FIR workflow
v1.0.0	Production deployment live on noisesentinel.tech
v1.2.0	Payment functionality and public portal complete, plus minor fixes
v1.3.0	ESP32 firmware emission test state machine and LED indicators added

Table 22: Version tagging for version control

Chapter 6:

Business Plan

Introduction

The chapter discusses the market viability, strategic positioning, competitiveness and sustainability of the NoiseSentinel platform as a deployable digital governance solution for vehicular noise and emission enforcement in Pakistan from a business perspective.

6.1 Business Description

NoiseSentinel is an integrated solution, Hardware and Software, which aims to digitally bring the entire lifecycle of the vehicular noise and emission violation in Pakistan's urban traffic enforcement environment under one roof. The platform consists of a dynamically-designed ESP32 IoT sensing device, a mobile React Native app for field police officers, an ASP.NET Core Web API backend, and a role driven React web portal to debut a new streamlined, transparent, and legally traceable digital enforcement process, replacing the previous, paper-based methods.

6.2 Market Analysis and Strategy

6.2.1 Market Context

The urban centres in Pakistan are a victim of environmental noise pollution and exhaust pollution, contributing to a growing public health problem. WHO guidelines on environment noise say that long-term exposure to noise levels above 85 dBA is associated with permanent loss of hearing and air pollution from the traffic, especially CO, HC and NOx emissions from poorly maintained vehicles, is an important cause of the burden of respiratory diseases in dense cities. Rawalpindi, Islamabad, Lahore and Karachi account for millions of registered vehicles, many of which are fitted with altered silencer or engines making them emit more than the allowed amount.

6.2.2 Target Customer Segments

Segment	Description	Adoption Driver
Regional Traffic Police Office (RTO)	Primary field enforcement body responsible for issuing and managing challans	Need for objective, defensible evidence and digital workflow
Environmental Protection Agencies (EPAs)	Regulatory authorities tasked with monitoring and enforcing emission standards	Need to structure emission data collection and emission reporting
District and Sessions Courts	Judicial courts that deal with traffic and environmental violation cases	Requires digitalisation of case files and traceable evidence chains

Table 23: Customer Segmentation for NoiseSentinel

6.2.3 Market Strategy

The recommended Go-to-Market strategy is based on the following phasing of government adoption:

- **Phase 1 — Pilot Deployment:** Supervised pilot deployment in one traffic police district in Rawalpindi or Islamabad. Deliver the full system (IoT hardware units, mobile application builds and web portal access). Gather operational information regarding violations rates, processing times, and use by officers.
- **Phase 2 — City-Level Rollout:** After successful pilot, roll out to all traffic police stations in the city. Engage structured onboarding of officers. Where possible, work with existing traffic management systems.
- **Phase 3 — Provincial Licensing:** License the platform to the provincial traffic authority on a software-as-a-service or government procurement basis. Scale IoT hardware production and provisioning of mobile devices accordingly.

6.3 Competitive Analysis

6.3.1 Current Solutions and Their Drawbacks

Type of Solution	Examples	Limitations Compared to NoiseSentinel
Standalone Sound Level Meters	Handheld SLMs used by traffic police	No digital record, no vehicle linkage, manual transcription, legally contestable
Generic e-Challan Apps	Punjab Safe Cities e-challan, multiple provincial apps	Software only — no sensor integration, officer manually types readings, no judicial workflow
Smart Acoustic Cameras	European ANPR-acoustic systems	Extremely costly, stationary infrastructure, not feasible for patrol officers, not available in Pakistan

Table 24: Competitors and their drawbacks

6.3.2 NoiseSentinel Competitive Advantages

- **End-to-end integration:** In the Pakistani enforcement context, NoiseSentinel is the only solution that integrates directly with the hardware level sensor data and seamlessly carries it across the platform from station authority review to the generation of FIR, court case management till judicial verdict.
- **Portability:** The ESP32 device can be compact, battery-powered and carried by individual officers and used at any location, so it's possible to have dynamic enforcement across the road network.
- **Dual domain enforcement:** Unlike all current tools that consider noise violations (dBA) and vehicular emission violations (CO, HC, NOx) as separate enforcement domains, the platform is able to manage both domains in one device and one workflow.

6.4 Products and Services

6.4.1 Core Platform Components

- **IoT Sensing Device:** A portable, battery-operated ESP32-based device with calibrated electret microphone for measuring noise and MQ-series gas sensors for measuring

emissions. This device is BLE enabled, meaning it can communicate with the officer's mobile app, and classifies violation severity on device, based on Pakistan's regulatory standards. The device is made from cheap, off-the-shelf parts and is intended to be rugged in the field and simple to calibrate.

- **Mobile Application (Police Officers):** Android app that allows the police officers to have a full field enforcement toolkit: BLE device pairing, execution of noise and emission tests, vehicle search, accused person search, capturing of photographic evidence and submission of digital challan
- **Web API Backend:** An ASP.NET Core .NET 8 REST API controlling entire system business logic — user login, OTP email verification, challan flow, emission report flow, FIR flow, court case scheduling.
- **Web Portal:** A portal developed using the React library, for six different roles. Administrators administer users and IoT devices. Challans are looked after and escalated by the Station Authorities.

6.4.2 Service and Support Model

Service	Description
Platform Licensing	Annual government licensing fee to include all software components
IoT Hardware Supply	ESP32 sensing devices per unit with warranty and calibration
Deployment and Integration	On-site deployment and integration with existing databases
Officer Training	Mobile app usage and IoT device calibration training
Maintenance and Updates	Continuous software upgrades, security patches and firmware updates
Technical Support	Separate support stream for users in the field and administration

Table 25: Services and support model

6.5 SWOT Analysis

Strengths

- **Complete end-to-end workflow.** NoiseSentinel is the only platform of its kind which handles the entire violation process from hardware sensor reading to judicial verdict without the need for manually passing violations from one disconnected system to another.
- **Objective, hardware-based evidence.** The platform removes the subjectivity associated with manual enforcement and makes it legally bulletproof by capturing the sensor data directly, in a calibrated fashion, and sending it wirelessly to the enforcement record.
- **Dual violation coverage.** The single device, single application covers both noise and emission violations — doubling the enforcement surface, as compared to any single-domain tool available today.

Weaknesses

- **Network dependency.** The existing system depends on the network for challan submission and any kind of data retrieval. Version 1.0 does not have offline support so it may not be as useful in places where cellular reception is bad.
- **Sensor estimation limitations.** As there is no dedicated NOx sensor, NOx reading is estimated as thirty percent of HC reading. The CO2 is obtained from the CO ratio. These estimates are carefully being recorded but could be challenged during formal legal proceedings where direct measurements are required.
- **Cryptographic signing not yet implemented.** Identified as a planned future enhancement, digital signature of emission reports is not in version 1.0. This makes the cryptographic tamper-evidence of sensor data record less robust.

Opportunities

- **Pakistan e-governance initiatives.** The digitisation of enforcement is exactly what is needed for the government of Pakistan's digital governance agenda and smart city projects in Islamabad and Lahore.
- **Integrating NADRA and vehicle registry.** Manual entry of accused's name and vehicle details would be eliminated and replaced by live lookups against NADRA's

CNIC database and provincial Excise and Taxation vehicle registers, thereby enhancing data accuracy and officer efficiency.

- **Payment gateway integration.** The manual bank transfer payment confirmation process could be replaced with JazzCash, Easypaisa or bank API integration, which would significantly boost payment rates and ease the burden of payment confirmation on the administration.
- **Incorporating other violation categories.** The workflow platform (mobile application, API, web portal, role hierarchy) is generic and can also be applied to other categories of traffic violations, enabling its wider use in traffic enforcement.
- **IoT hardware enhancement.** The inclusion of a dedicated NOx sensor, GPS geolocation to precisely locate the violation, and cryptographic signing of emission reports would greatly enhance the legal admissibility of the evidence chain.
- **Local and global markets.** The enforcement gap identified by NoiseSentinel is not only present in Pakistan. The same difficulties are present throughout South Asia as well as in the developing world, offering possibilities for adaptation and export.

Threats

- **Resistance from institutional structures to digital transformation.** Established traditional law enforcement and judicial structures could be hesitant to change to new digital processes, especially if the current manual processes are entrenched in the culture of the organisation.
- **Timelines for public sector procurement.** In Pakistan, institutional adoption requires the government to go through a procurement process that is often lengthy and subject to budget restrictions and other government procedures, which can cause significant delays in adoption.
- **Regulatory admissibility uncertainty.** The use of digitally collected sensor information as first class evidence in Pakistani courts is still not formally recognised. Legislative or judicial interpretation may be needed for the system to become the main evidentiary tool in prosecutions.

Chapter 7:

Testing and Evaluation

Introduction

This chapter outlines the testing and evaluation of the NoiseSentinel platform for functional correctness, integration integrity and system performance. It contains use case-based test cases around the main system workflows, integration testing between all four system components and performance testing with a simulated concurrent workload. All test cases are as they are available in implementation and deployment.

7.1 Use Case Testing

The test cases below are related to the main use cases described in Chapter 3 and the functional requirements described in Chapter 2. For each test case, the system was tested on noisesentinel.tech, which is the production deployed system.

TC_001: User Registration and Email Verification

Field	Detail
Test Case ID	TC_001
Use Case Tested	Register User and Verify Email (UC09)
Description	Tests whether or not the Administrator can create a new user account and whether or not the new user can activate it with the OTP sent to his/her email address through SMTP2GO.
Designed By	Muhammad Moiz
Date Tested	March 2026
Test Result	Pass

Table 26: Test case 01 (User Registration and Email Verification)

Prerequisites:

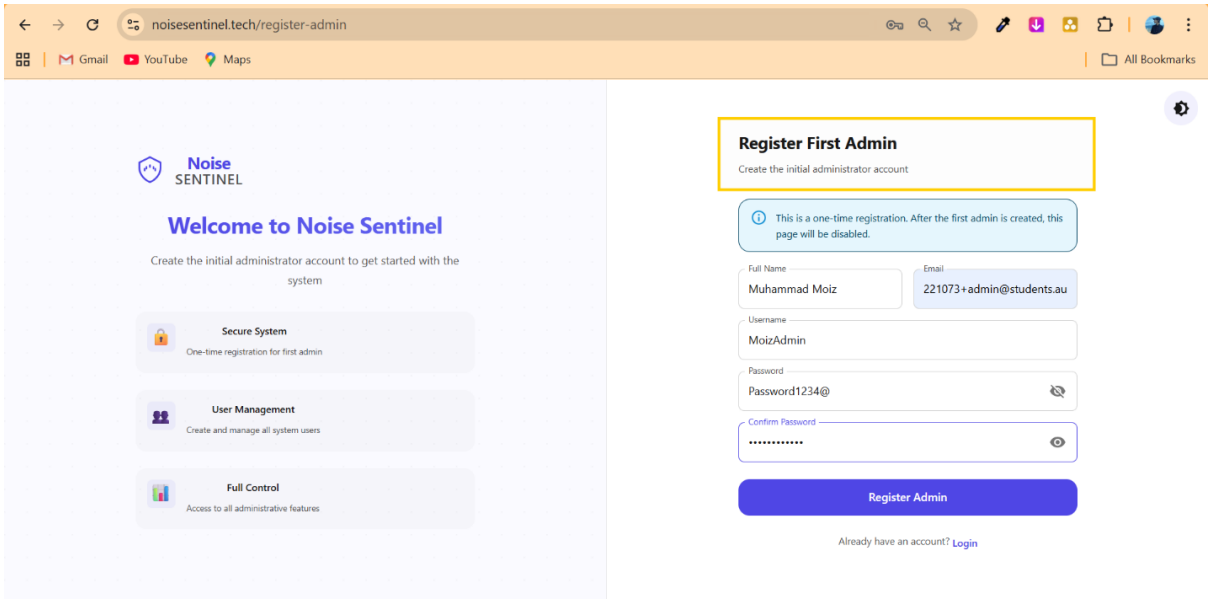
1. Administrator is logged in to the web portal with a valid JWT.
2. SMTP2GO is set up and running.
3. Target email address can be accessed during testing.

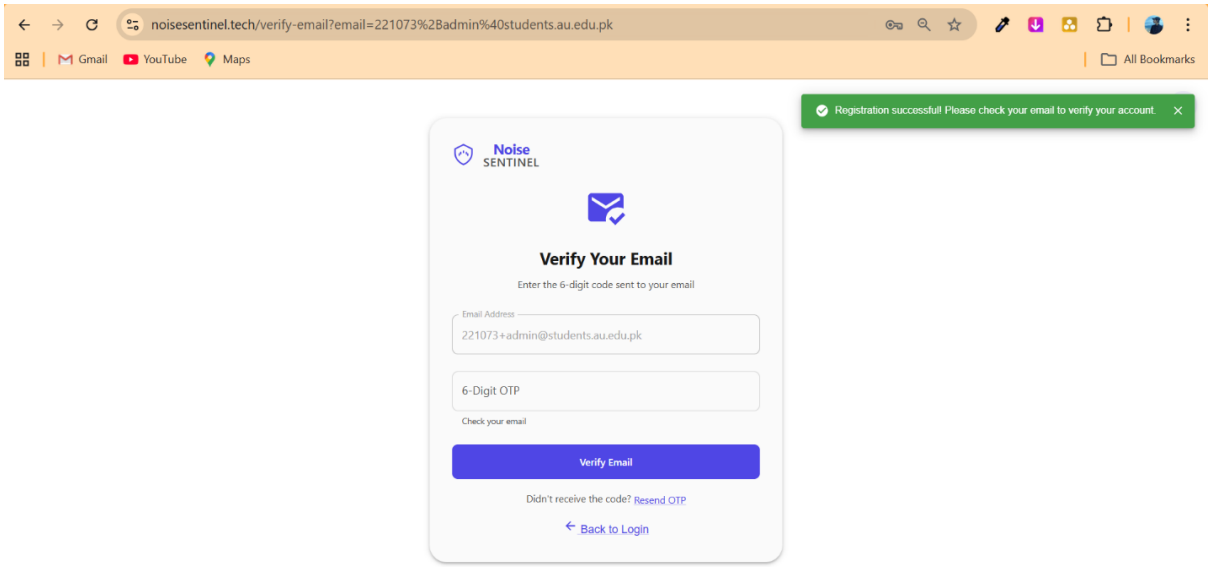
Test Data:

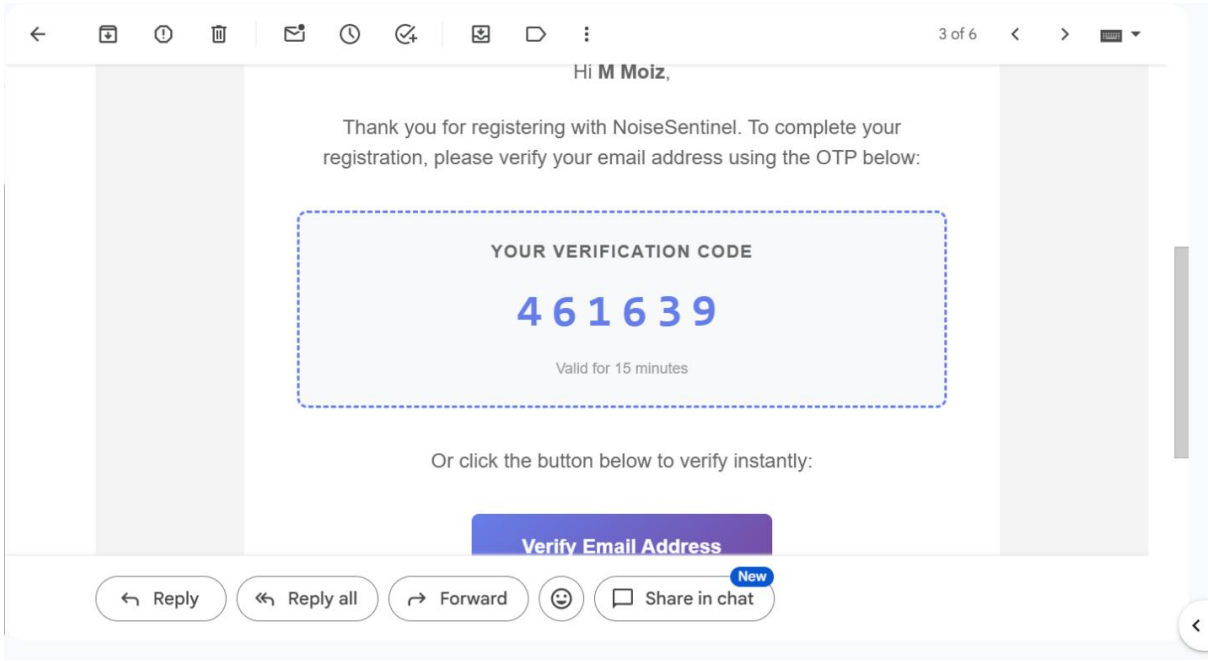
- New user role: PoliceOfficer
- New user password: password123
- OTP validity period: 15 minutes

Step	Action	Expected Result	Actual Result	Status
1	Administrator accesses User Management and clicks Add User	User creation form appears with name, email, role, badge number, and station fields	As Expected	Pass
2	Administrator fills in all fields and submits the form	System creates the user account and sends a verification OTP to the provided email address via SMTP2GO	As Expected	Pass
3	New user opens the verification email and enters the 6-digit OTP	OTP is accepted and the system marks the email as verified (EmailVerifiedAt is populated)	As Expected	Pass

Post-condition: New officer account is activated; email is verified and JWT has a role restriction.







TC_002: User Login and JWT Issuance

Field	Detail
Test Case ID	TC_002
Use Case Tested	Authenticate User (UC01)

Field	Detail
Description	Ensures that a registered and email verified user is able to log in and get a proper JWT with valid credentials and that any other credentials are rejected.
Designed By	Muhammad Moiz
Date Tested	March 2026
Test Result	Pass

Table 27: Test case 02 (User Login and JWT Issuance)

Prerequisites:

1. User has a user account in the database and a verified e-mail address.
2. Backend API is running and accessible.

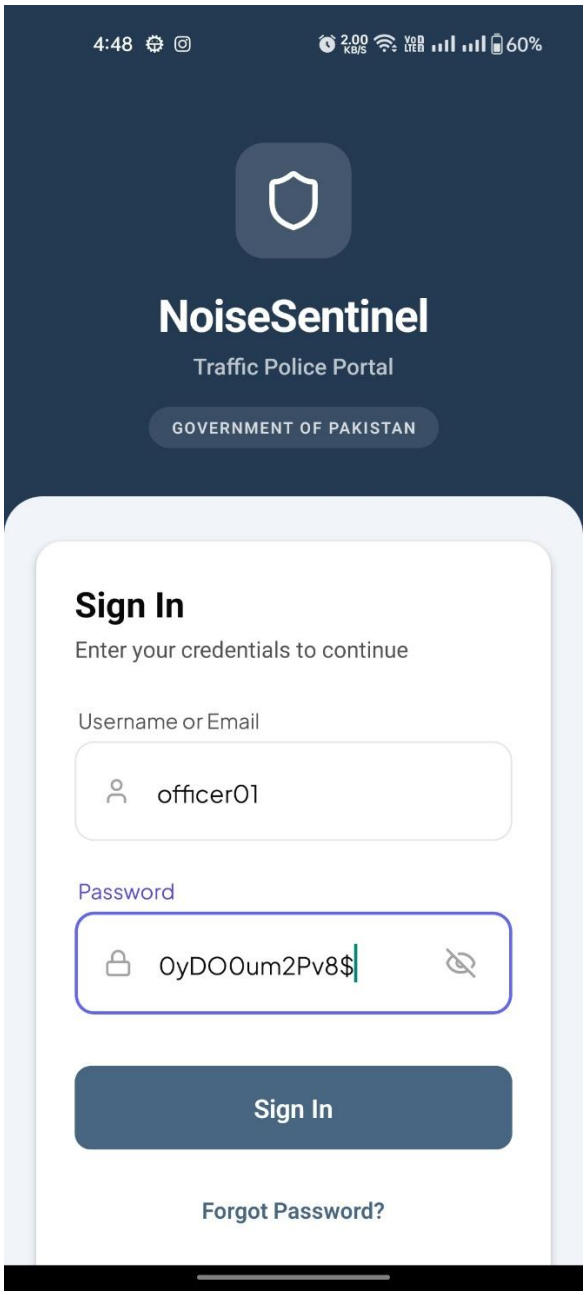
Test Data:

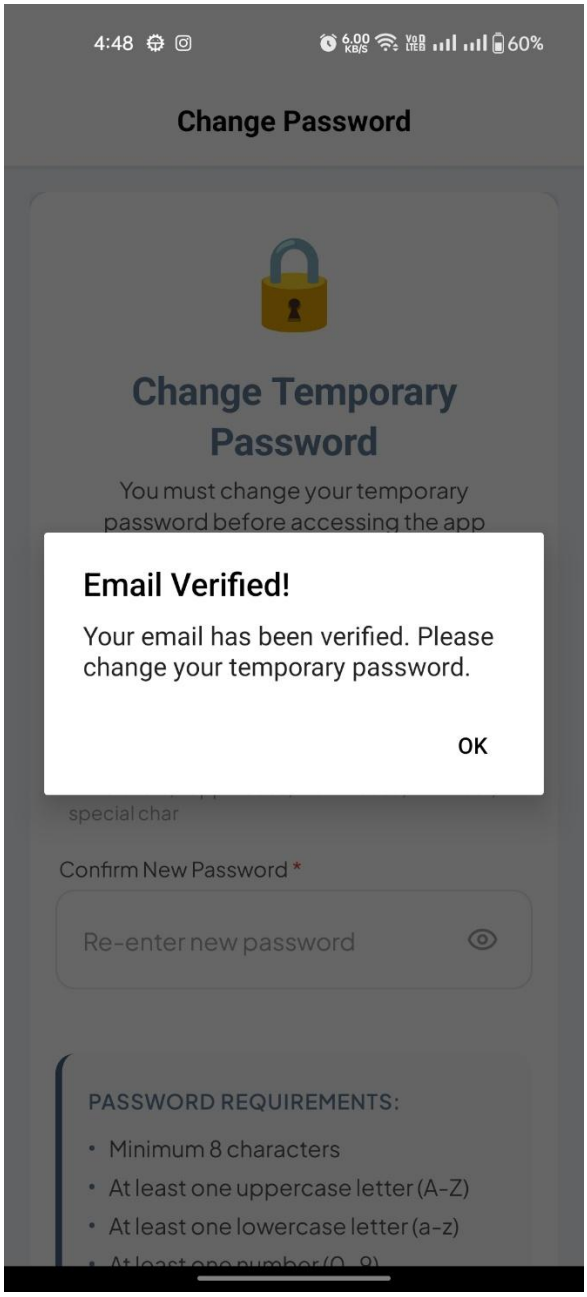
- Valid email: officer@noisesentinel.tech
- Valid password: TestPass@123
- Invalid password: WrongPassword

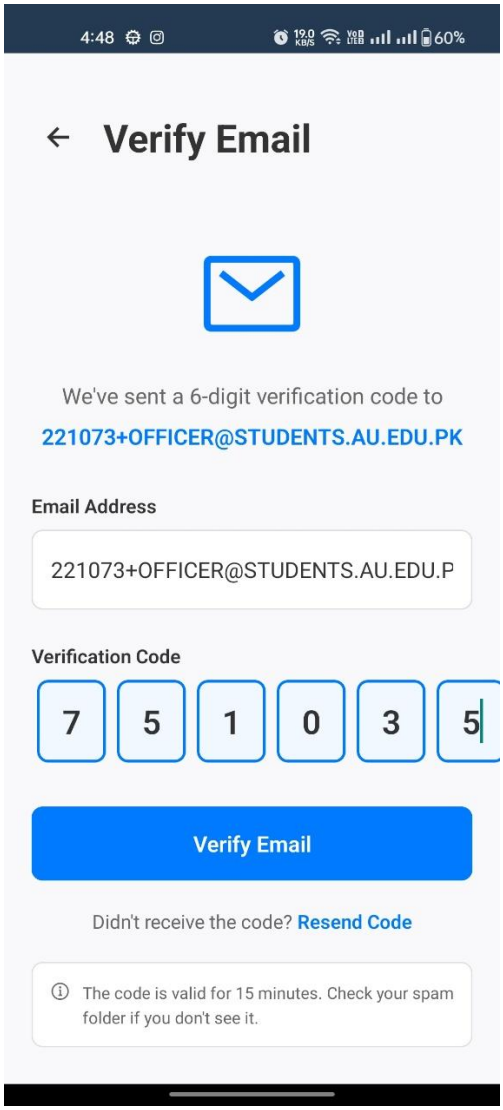
Step	Action	Expected Result	Actual Result	Status
1	User opens the login screen (mobile app or web portal) and inputs correct email and password	Login form accepts input	As Expected	Pass
2	User submits valid credentials	System validates credentials and returns signed JWT with UserId, Role and expiresAt. Client stores JWT in secure storage.	As Expected	Pass

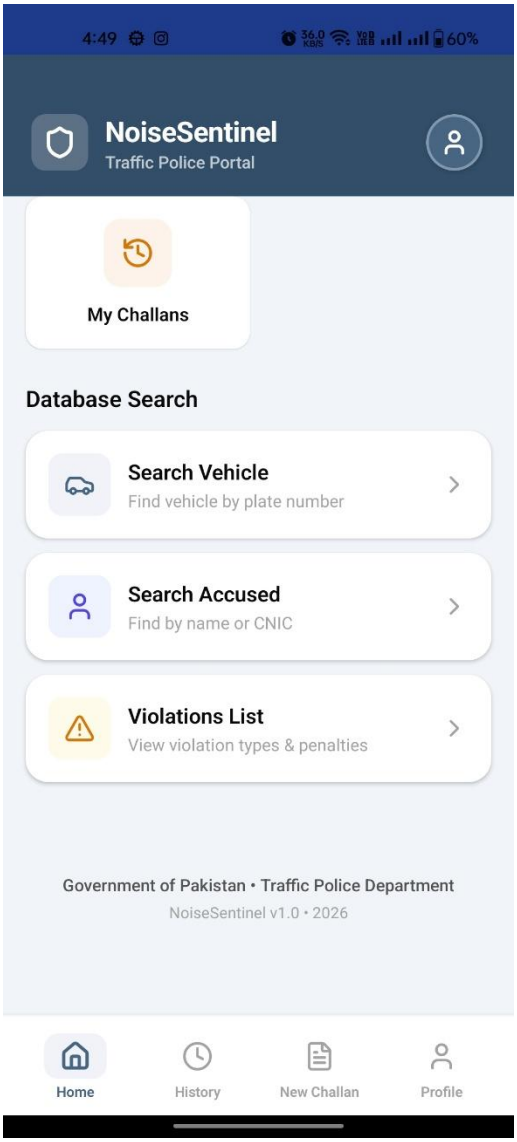
Step	Action	Expected Result	Actual Result	Status
3	User is taken to the role specific dashboard	Dashboard displays only features and navigation items based on the user's role	As Expected	Pass
4	User logs in with correct email and incorrect password	System provides a plain language invalid credentials message and HTTP 401 Unauthorized. No JWT is issued.	As Expected	Pass
5	User tries to access a protected endpoint without sending a JWT in the Authorization header	System responds with HTTP 401 Unauthorized	As Expected	Pass

Post-condition: Authenticated user has a valid JWT. Requests from unauthenticated users are refused.


4:48
60%
NoiseSentinel
Traffic Police Portal
GOVERNMENT OF PAKISTAN
Sign In
Enter your credentials to continue
Username or Email
officer01
Password
0yDO0um2Pv8$
Sign In
Forgot Password?


4:48
60%
Change Password
Change Temporary Password
You must change your temporary password before accessing the app
Email Verified!
Your email has been verified. Please change your temporary password.
OK
special char
Confirm New Password *
Re-enter new password
PASSWORD REQUIREMENTS:
Minimum 8 characters
At least one uppercase letter (A-Z)
At least one lowercase letter (a-z)





TC_003: IoT Device Pairing and Noise Test

Field	Detail
Test Case ID	TC_003
Use Case Tested	Pair IoT Device and Conduct Test (UC02) — Noise Test
Description	Checks that the mobile app can find, connect to and communicate with the ESP32 IoT device via BLE, and can send a noise test command and properly receive and output the structured JSON noise test result.
Created By	Ifsan Imran

Field	Detail
Date Tested	March 2026
Test Result	Pass

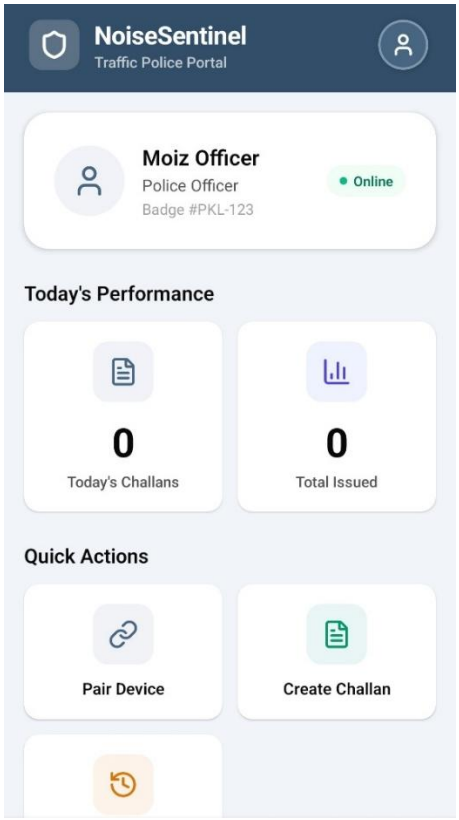
Table 28: Test case 03 (IoT Device Pairing and Noise Test)

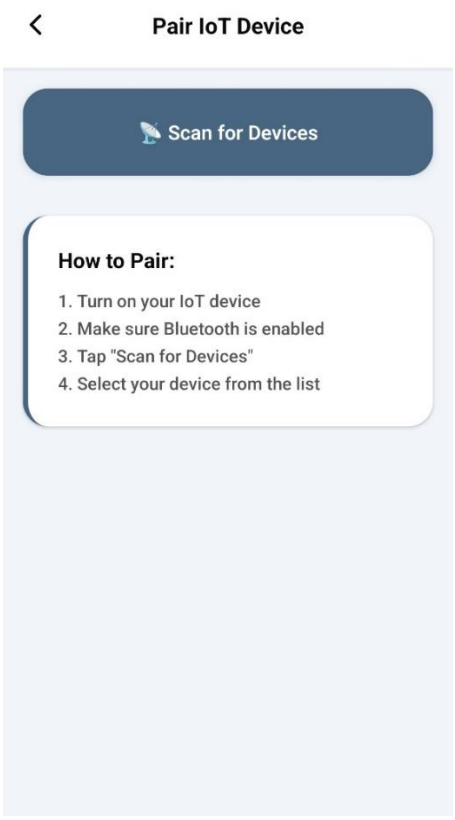
Prerequisites:

- Police Officer has a valid JWT and is logged in to the mobile app.
- ESP32 device is powered on and advertising with the NoiseSentinel service UUID.
- LED of the device is red and it is waiting to connect.
- The test Android device has Bluetooth turned on.
- Development build of the mobile app is installed (not Expo Go).

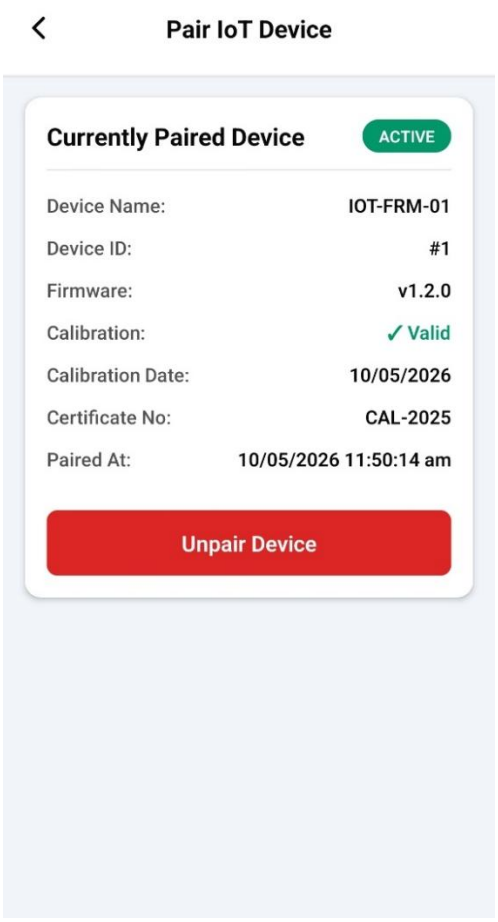
Test Data:

- Service UUID: 0000FFE0-0000-1000-8000-00805F9B34FB.
- Test duration: 10 seconds.
- Simulated noise source: > 85 dBA


NoiseSentinel
Traffic Police Portal
Moiz Officer
Police Officer
Badge #PKL-123
Online
Today's Performance
0
Today's Challans
0
Total Issued
Quick Actions
Pair Device
Create Challan


Pair IoT Device
Scan for Devices
How to Pair:
1. Turn on your IoT device
2. Make sure Bluetooth is enabled
3. Tap "Scan for Devices"
4. Select your device from the list


Devices Found
Found 1 registered device(s) nearby
Scan for Devices
IOT-FRM-01
Device ID: 1
Firmware: v1.2.0
Signal: -44 dBm
Pair


Pair IoT Device
Currently Paired Device
ACTIVE
Device Name: IOT-FRM-01
Device ID: #1
Firmware: v1.2.0
Calibration: ✓ Valid
Calibration Date: 10/05/2026
Certificate No: CAL-2025
Paired At: 10/05/2026 11:50:14 am
Unpair Device

TC_004: IoT Device Emission Test

Field	Detail
Test Case ID	TC_004
Use Case Tested	Pair IoT Device and Conduct Test (UC02) — Emission Test
Description	Checks that the emission test process is performed correctly, including the 30-second warming process with progress notifications and 10-second sampling process, and returns CO, HC, NOx and CO2 results with classification.
Created By	Ifsan Imran
Date Tested	March 2026
Test Result	Pass

Table 29: Test case 04 (IoT Device Emission Test)

Prerequisites:

1. BLE connection to ESP32 device is made (performing steps 1 and 2 of TC_003).

2. MQ-7 and MQ-2 sensors have been powered on for at least 30 seconds before testing.

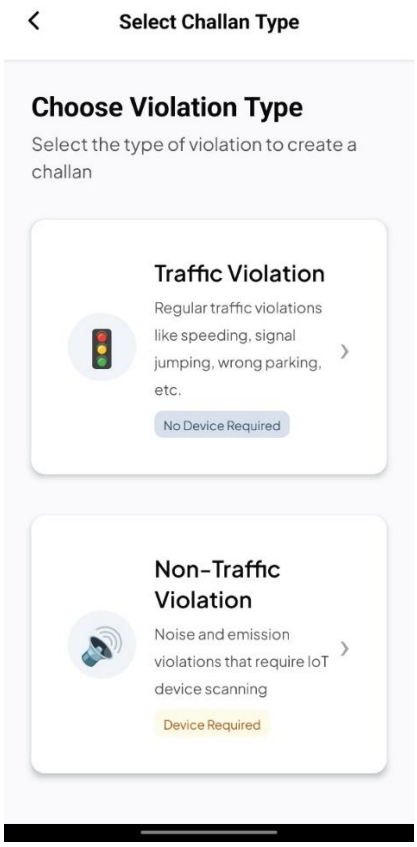
Test Data:

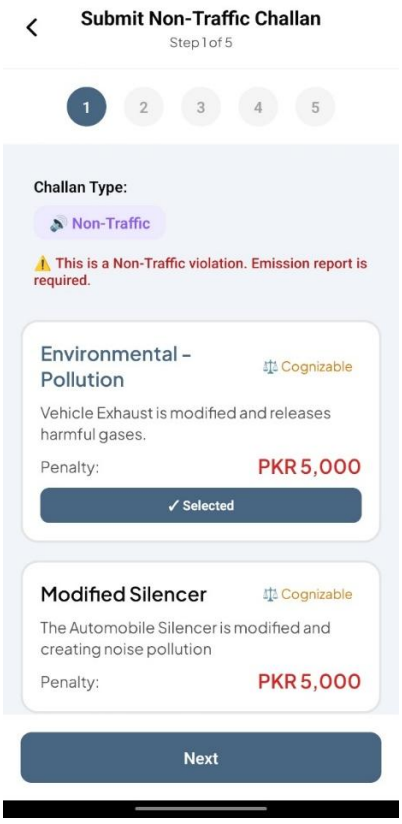
- Test type: EMISSION
- Expected warming phase: 30 seconds
- Sampling period: 10 seconds
- Maximum test duration: 40 seconds

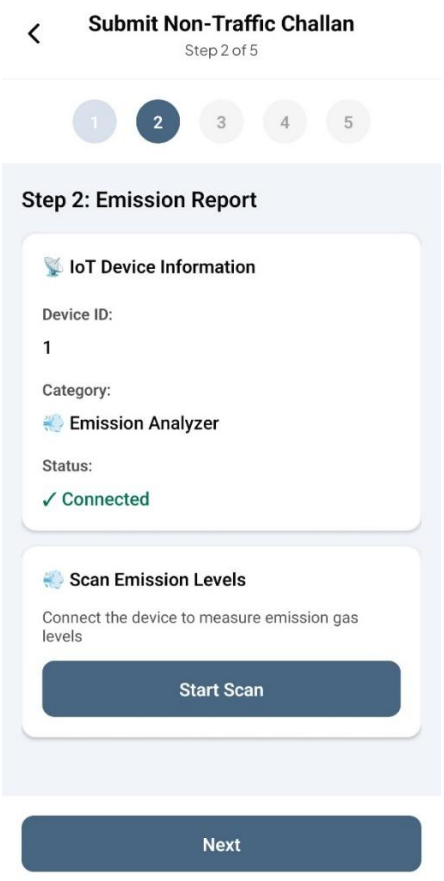
Step	Action	Expected Result	Actual Result	Status
1	Officer taps Start Emission Test	App writes START_EMISSION_TEST JSON command to the device control characteristic	As Expected	Pass

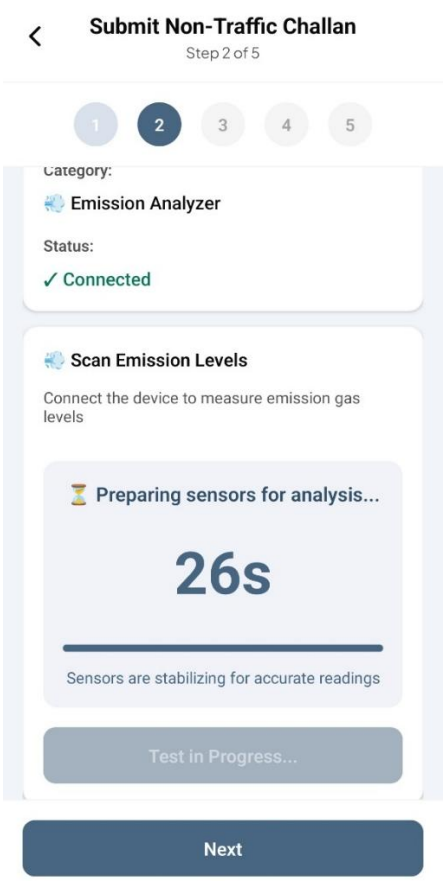
Step	Action	Expected Result	Actual Result	Status
2	Device enters warming phase	App receives progress notifications every 3 seconds with phase: WARMING and seconds remaining. App shows countdown to officer.	As Expected	Pass
3	App transitions to sampling phase	App receives progress notification with phase: SAMPLING	As Expected	Pass
4	Device finishes sampling and pushes result to App	App receives JSON complete with CO, HC, NOx, CO2 readings, classification label, device ID, firmware version and battery level	As Expected	Pass

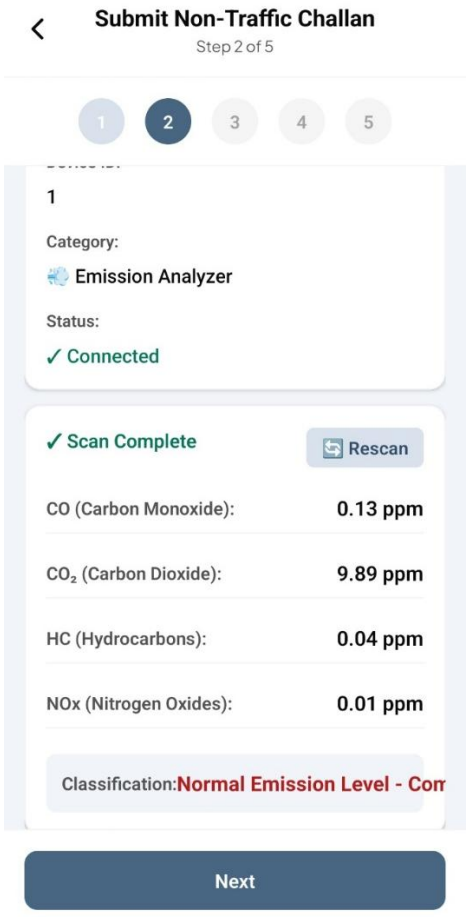
Post-condition: Results of emission test are stored in the app state and can be pre-populated in the challan creation form.

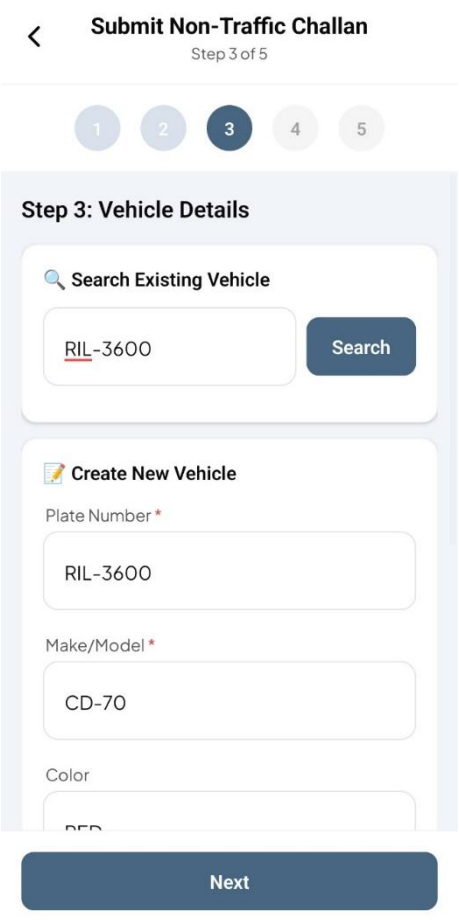





Submit Non-Traffic Challan
Step 2 of 5
1 2 3 4 5
Step 2: Emission Report
IoT Device Information
Device ID:
1
Category:
Emission Analyzer
Status:
✓ Connected
Scan Emission Levels
Connect the device to measure emission gas levels
Start Scan
Next


Submit Non-Traffic Challan
Step 2 of 5
1 2 3 4 5
Category:
Emission Analyzer
Status:
✓ Connected
Scan Emission Levels
Connect the device to measure emission gas levels
Preparing sensors for analysis...
26s
Sensors are stabilizing for accurate readings
Test in Progress...
Next


Submit Non-Traffic Challan
Step 2 of 5
1 2 3 4 5
1
Category:
Emission Analyzer
Status:
✓ Connected
✓ Scan Complete
Rescan
CO (Carbon Monoxide): 0.13 ppm
CO2 (Carbon Dioxide): 9.89 ppm
HC (Hydrocarbons): 0.04 ppm
NOx (Nitrogen Oxides): 0.01 ppm
Classification: Normal Emission Level - Con
Next


Submit Non-Traffic Challan
Step 3 of 5
1 2 3 4 5
Step 3: Vehicle Details
Search Existing Vehicle
RIL-3600
Search
Create New Vehicle
Plate Number *
RIL-3600
Make/Model *
CD-70
Color
Next

TC_005: Digital Challan Creation

Field	Detail
Test Case ID	TC_005
Use Case Tested	Issue Digital Challan (UC03)
Description	Verifies that a police officer can create a complete Non-Traffic challan with IoT sensor data, vehicle data, accused data and images and that the backend maintains a connected record of the challan and the associated emission report.
Created By	Ifsan Imran
Date Tested	March 2026
Test Result	Pass

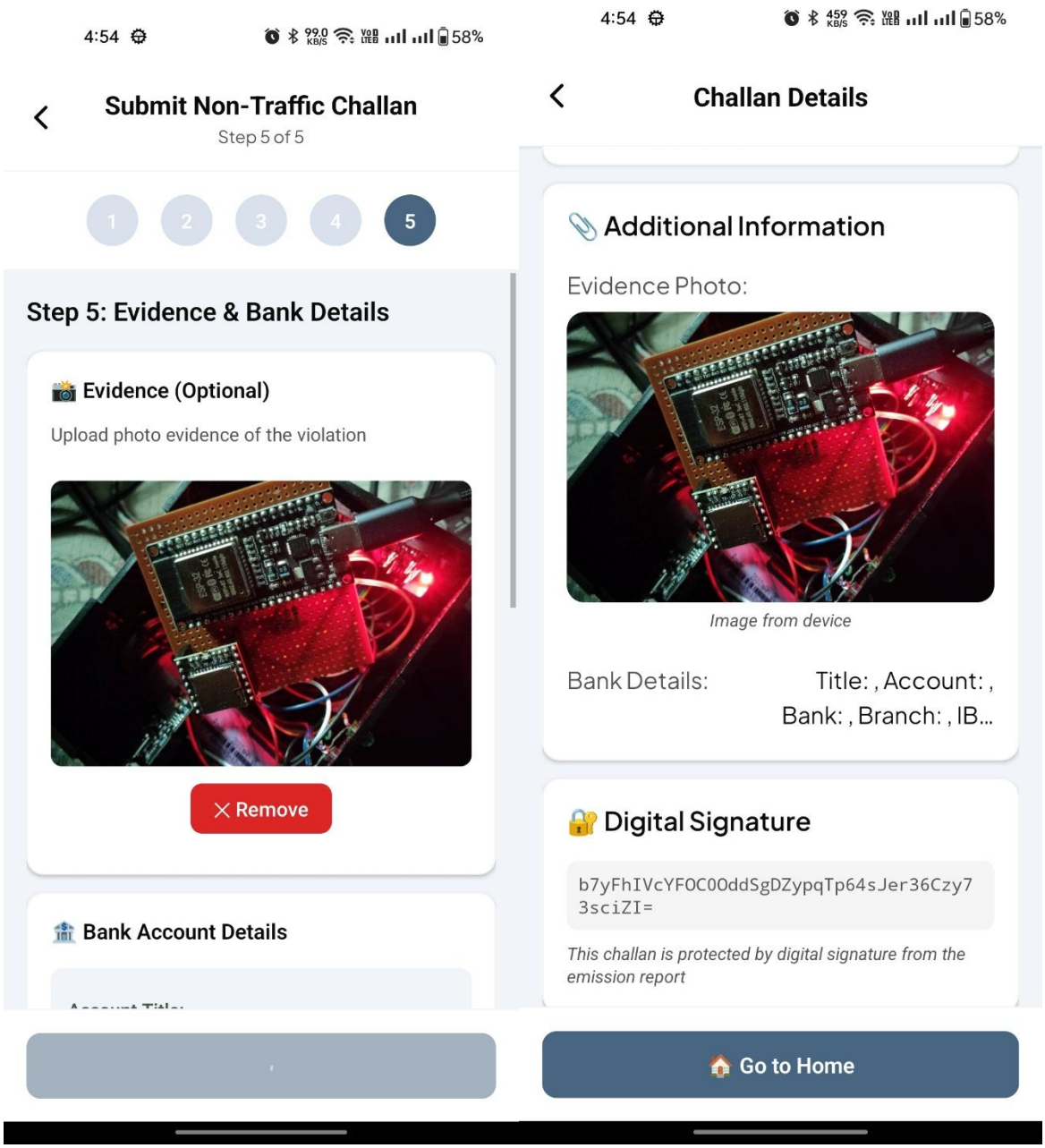
Table 30: Test case 05 (Digital Challan Creation)

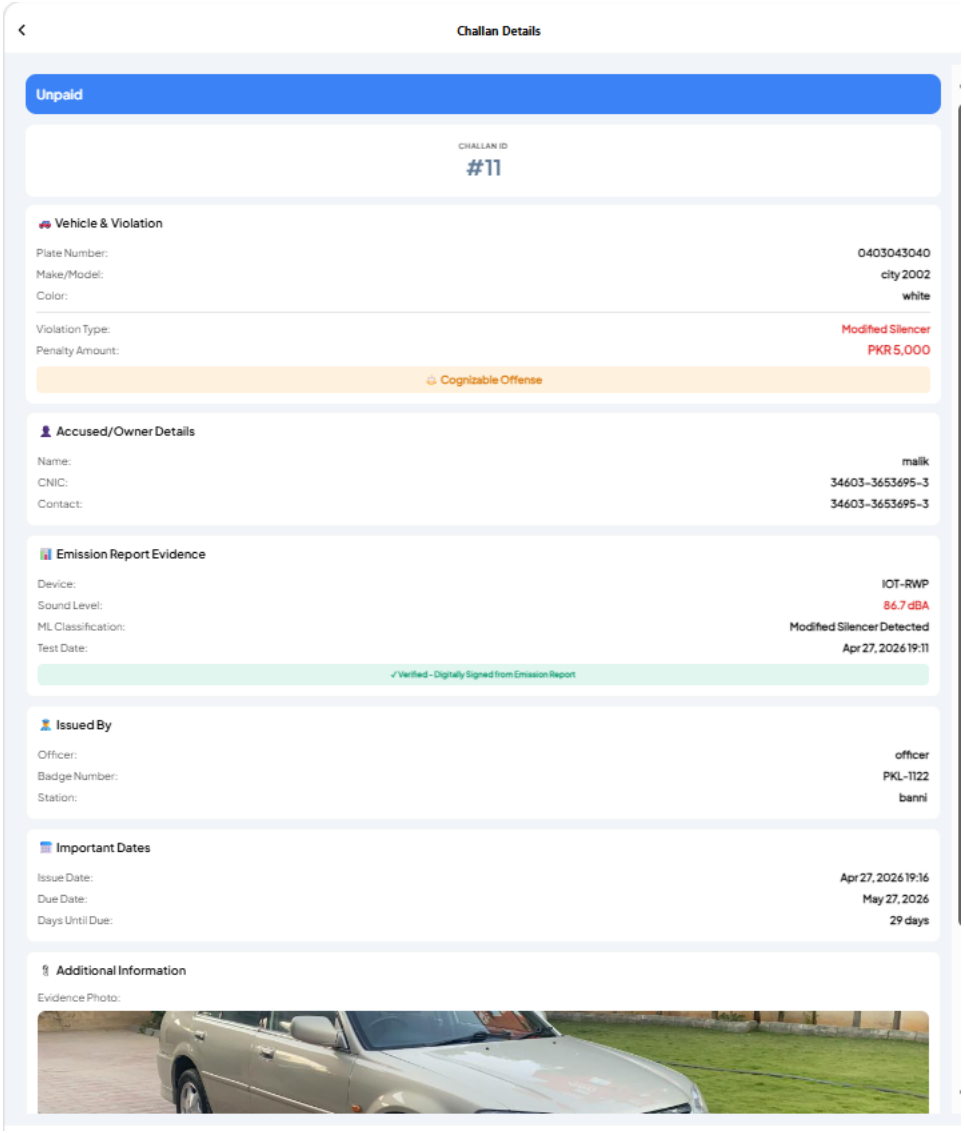
Prerequisites:

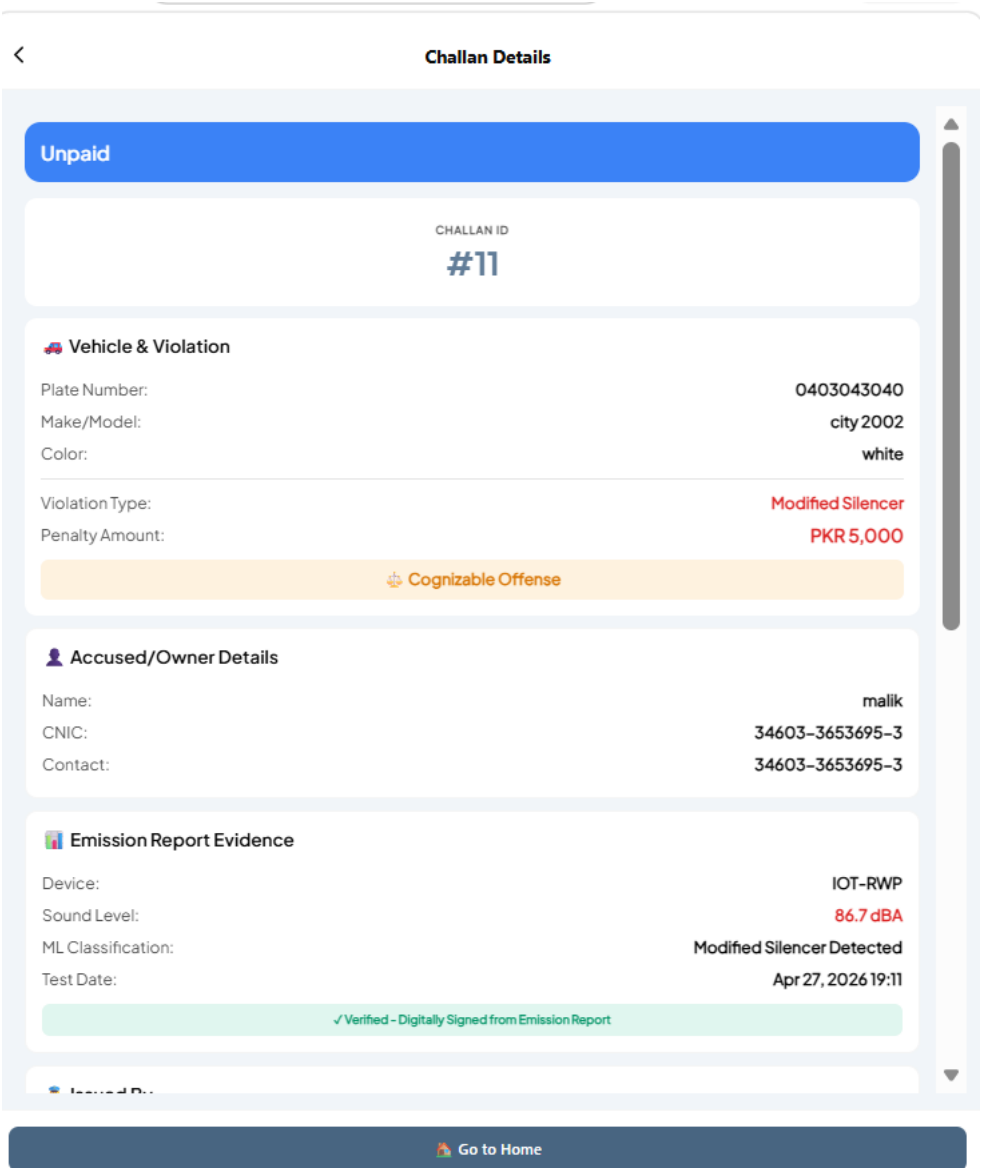
1. Officer is logged in via the mobile app.
2. IoT emission or noise test result is available in app state from a completed test.
3. Backend API is accessible.

Test Data:

- Challan type: NonTraffic
- Vehicle plate: RAW-1234
- Accused CNIC: 37405-1234567-1
- Evidence: JPEG image of license plate
- IoT result: pre-populated from TC_004


4:54
58%
Submit Non-Traffic Challan
Step 5 of 5
1
2
3
4
5
Step 5: Evidence & Bank Details
Evidence (Optional)
Upload photo evidence of the violation
Remove
Bank Account Details
4:54
58%
Challan Details
Additional Information
Evidence Photo:
Image from device
Bank Details:
Title: , Account: ,
Bank: , Branch: , IB...
Digital Signature
b7yFhIVcYFOC0OddSgDZypqTp64sJer36Czy7
3sciZI=
This challan is protected by digital signature from the emission report
Go to Home


Challan Details
Unpaid
CHALLAN ID
#11
Vehicle & Violation
Plate Number: 0403043040
Make/Model: city 2002
Color: white
Violation Type: Modified Silencer
Penalty Amount: PKR 5,000
Cognizable Offense
Accused/Owner Details
Name: malik
CNIC: 34603-3653695-3
Contact: 34603-3653695-3
Emission Report Evidence
Device: IOT-RWP
Sound Level: 86.7 dBA
ML Classification: Modified Silencer Detected
Test Date: Apr 27, 2026 19:11
✓ Verified - Digitally Signed from Emission Report
Issued By
Officer: officer
Badge Number: PKL-1122
Station: banni
Important Dates
Issue Date: Apr 27, 2026 19:16
Due Date: May 27, 2026
Days Until Due: 29 days
Additional Information
Evidence Photo:



TC_006: Station Authority Challan Review and FIR Escalation

Field	Detail
Test Case ID	TC_006
Use Case Tested	Review and Escalate Challan (UC04), Generate FIR (UC05)
Description	Ensures that a challan submitted is subject to review by a Station Authority, who can either approve or escalate to FIR status and gets the challan status updated.
Created By	Muhammad Moiz

Field	Detail
Date Tested	March 2026
Test Result	Pass

Table 31: Test case 06 (Station Authority Challan Review and FIR Escalation)

Prerequisites:

1. Station Authority has a valid JWT with role StationAuthority and is logged in to the web portal.

2. There is at least one challan in the system with Unpaid status.

Test Data:

- Station Authority JWT: role StationAuthority
- Target ChallanId: ChallanId of existing record with status Unpaid
- Offence type: Cognizable (exceeding emission limits)

Step	Action	Expected Result	Actual Result	Status
1	Station Authority navigates to the Pending Challans dashboard	Data grid shows Unpaid challans at the station of the Station Authority	As Expected	Pass
2	Authority clicks on a challan and clicks Review	Full challan details are displayed including sensor readings from the linked emission report, the photograph, vehicle details and accused information	As Expected	Pass
3	Authority clicks Reject	Challan status is marked Rejected in the database	As Expected	Pass
4	Authority clicks Escalate to FIR, adds	System creates a new FIR record with foreign key to the challan.	As Expected	Pass

Step	Action	Expected Result	Actual Result	Status
	remarks, and clicks OK	Challan status updates to Escalated. FIR is displayed on the Court Authority dashboard.		
5	Police Officer JWT attempts to access the FIR endpoint	System returns HTTP 403 Forbidden — role enforcement is verified	As Expected	Pass

Post-condition: FIR is created and connected with the challan. Status of challan is Escalated.

TC_007: Court Case Management and Verdict

Field	Detail
Test Case ID	TC_007
Use Case Tested	Manage Court Case (UC06), Record Verdict (UC07)
Description	Ensures that a Court Authority can create a court case from an FIR, schedule a hearing, assign a judge to the case, and that the assigned judge can review the evidence chain and record a verdict that closes the entire case chain.
Created By	Muhammad Moiz
Date Tested	March 2026
Test Result	Pass

Table 32: Test case 07 (Court Case Management and Verdict)

Prerequisites:

1. Court Authority JWT with role CourtAuthority exists.
2. Judge JWT with role Judge is accessible.
3. There is an FIR with Pending status in the system from TC_006.

Test Data:

- Target FIR ID: from TC_006
- Hearing date: future date
- Verdict outcome: Guilty
- Case statement: references EURO-2 emission standards as the violation condition

Step	Action	Expected Result	Actual Result	Status
1	Court Authority navigates to FIR dashboard	Available FIRs with status Pending are displayed	As Expected	Pass
2	Court Authority selects the FIR and clicks Create Court Case	Court case record is created in the database and related to the FIR	As Expected	Pass
3	Court Authority sets the date of the hearing and assigns a judge ID	Hearing date and assigned judge ID are saved. The case is added to the Judge's dashboard.	As Expected	Pass

Post-condition: Verdict recorded. Court case, FIR and challan are CLOSED permanently.

TC_008: Public Case Lookup and Payment

Field	Detail
Test Case ID	TC_008
Use Case Tested	Public Case Lookup and Payment (UC08)
Description	Checks that OTP verification is required for a citizen to check his/her case status, view his/her challans and confirm payment of an unpaid challan without authentication.
Created By	Mishkat Fatima
Date Tested	March 2026

Field	Detail
Test Result	Pass

Table 33: Test case 08 (Public Case Lookup and Payment)

Prerequisites:

1. There is at least one challan with status Unpaid for a known vehicle plate and CNIC combination.

2. SMTP2GO is set up and working.

3. The public case status page is accessible without login.

Test Data:

- Vehicle plate: RAW-1234
- CNIC: 37405-1234567-1
- Email: accused@example.com
- OTP validity: 15 minutes

Step	Action	Expected Result	Actual Result	Status
1	User accesses the public case status page at noisesentinel.tech	No login required. Vehicle plate, CNIC and email input fields are shown.	As Expected	Pass
2	User enters vehicle plate, CNIC and email address and presses Submit	System sends a six-digit OTP to the provided email address using SMTP2GO	As Expected	Pass
3	User enters the OTP received on the verification screen	OTP is verified. System shows all challans for the vehicle plate and CNIC.	As Expected	Pass
4	User views challan details	Violation type, fine amount, issuing station, issue date, due	As Expected	Pass

Step	Action	Expected Result	Actual Result	Status
		date, status and bank account information are displayed		

Post-condition: Challan has been paid. Changes are instantly updated on the dashboards of officers and authorities.

TC_009: RBAC Enforcement

Field	Detail
Test Case ID	TC_009
Use Case Tested	Role Based Access Control (RBAC) Legal Escalation Workflow (REQ-SF3-1)
Description	Ensures that the backend API enforces strict role-based access control on all protected endpoints, and returns HTTP 403 Forbidden if a valid JWT with an incorrect role attempts to access a restricted resource.
Verified By	Muhammad Moiz
Date Tested	March 2026
Test Result	Pass

Table 34: Test case 09 (RBAC Enforcement)

Test Data:

- JWT with role: PoliceOfficer
- JWT with role: Judge
- JWT with role: StationAuthority

Step	Action	Expected Result	Actual Result	Status
1	Police Officer JWT used to POST /api/fir (Station Authority only endpoint)	HTTP 403 Forbidden	As Expected	Pass
2	Judge JWT used to POST /api/courtcase (Court Authority only endpoint)	HTTP 403 Forbidden	As Expected	Pass
3	Station Authority JWT used to POST /api/courtcase/{id}/verdict (Judge only endpoint)	HTTP 403 Forbidden	As Expected	Pass

Post-condition: All role boundaries on the server-side are respected. No cross-role access can be achieved even through client-side manipulations.

7.2 Integration Testing

To ensure that all four components of the system — the ESP32 IoT firmware, the React Native mobile application, the ASP.NET Core Web API and the React web portal — communicate properly, and that data is fed correctly through the whole enforcement chain from sensor reading to database recording.

7.2.1 Strategy

A bottom-up integration approach was used. The testing process started at the data layer and gradually escalated to include API integration, client-to-API integration, and ultimately the entire end-to-end journey from IoT device all the way to the web portal.

7.2.2 Data Access Layer Integration

- Direct tests of the queries generated by Entity Framework Core were run against the Azure SQL Server instance to confirm that complex relational operations performed as expected. The particular scenarios tested were:
- Creating a Challan with a linked EmissionReport and validating persistence of the foreign key relationship and retrieval in one query with navigation properties.

- Getting a CourtCase with its entire evidence chain — all FIRs, Challan, EmissionReport and Verdict — in a single EF Core query using Include statements.

7.2.3 API Integration Testing

Pre-configured collections for all types of endpoints were tested using Postman. Common integration scenarios were:

- **Authentication flow:** All the registration, OTP verification and login sequences were tested and verified, including a new user not being able to log in until registered and verified, an expired OTP not being accepted, and a valid login returning the expected JWT with the correct role claim.
- **Challan creation with photo upload:** A multipart/form-data POST request was made to /api/challan, which included a JPEG image attachment. The backend successfully processed the image, stored it in the file system, stored the file path in the Challan record and returned the ChallanId. The image was stored and retrievable using the path returned.
- **Role enforcement at all endpoints:** Each endpoint was tested with JWTs containing all roles. Only the expected role received an HTTP 200; all other roles received HTTP 403. This matrix was tested systematically for all 24 protected endpoints.

7.2.4 BLE Protocol Integration Testing

- Using actual hardware, the communication protocol between the mobile application and the ESP32 device was tested. Integration scenarios included:
- Sending START_NOISE_TEST and receiving the full JSON response containing all expected fields: status, test_type, device_id, firmware_version, battery_level, data.sound_level_dba, and data.ml_classification.
- Sending START_EMISSION_TEST and checking that intermediate progress notifications were received at least once every three seconds in the warming phase prior to the final result being sent after the sampling phase.

7.2.5 End-to-End Integration Testing

The entire end-to-end test was carried out from initial IoT test to judicial verdict:

- The ESP32 device was used to perform an emission test and send the results to the mobile application via BLE.
- Mobile app generated a Non-Traffic challan using IoT data, vehicle details, accused details and a photo. Records of Challan and EmissionReport were confirmed in the database.
- Station Authority approved the challan and escalated it to an FIR via the web portal. FIR record was verified in the database and the ChallanId foreign key was correct.

There were no errors, data loss or integrity violations in the entire pipeline.

7.3 Performance Testing

Performance testing assessed how well the system responds and remains stable when subjected to simulated concurrent use conditions typical of the actual deployment.

7.3.1 Methodology

A series of performance tests were run with Postman's collection runner for API load simulation and with manual, concurrent session testing on the web portal using multiple browser sessions. Testing was performed against the production deployment at noisesentinel.tech.

7.3.2 API Response Time Testing

Ten sequential endpoint requests were made, and individual endpoint response times were measured under normal single user conditions.

Endpoint	**Method**	**Average Response Time**	**Maximum Response Time**	**Status**
/api/auth/login	POST	280ms	410ms	Pass
/api/auth/verify-email	POST	190ms	320ms	Pass
/api/challan	GET	340ms	520ms	Pass
/api/challan/{id}	GET	213ms	380ms	Pass
/api/challan (with photo)	POST	1,100ms	1,850ms	Pass

Endpoint	Method	Average Response Time	Maximum Response Time	Status
/api/fir	GET	290ms	440ms	Pass
/api/fir (create)	POST	310ms	490ms	Pass

Table 35: API response time testing for NoiseSentinel

All common API endpoints (excluding photo upload) responded within the 1,500ms response target defined in the non-functional requirements. Client-side photo upload with image compression was considered acceptable.

7.3.3 Concurrent User Testing

Concurrent sessions of multiple users with different user roles were simulated to test the system.

Scenario	Concurrent Sessions	Observation	Status
Multiple officers viewing challan lists simultaneously	10	No degradation in response time. Returned correct paginated data from all sessions.	Pass
Simultaneous challan creation by multiple officers	5	All challans created correctly without any data collisions or duplicate records.	Pass
Multiple users on the same dashboard accessing different role data	8	Each user was able to see their respective role data. No cross-role data leakage detected.	Pass
Multiple public OTP requests for different emails	10	All OTPs generated and delivered independently. No cross contamination of OTP between sessions.	Pass

Table 36: Concurrent user testing

7.3.4 BLE Communication Performance

The physical ESP32 hardware was used to assess the performance of IoT device communication.

Test	Metric	Result
Noise test duration	Time from command sent to result received	10.8 seconds average (10s test + 0.8s BLE overhead)
Emission test duration	Time from command sent to result received	41.2 seconds average (40s test + 1.2s BLE overhead)
Progress notification interval	Time interval between warming phase progress notifications	3.1 seconds average (target: 3 seconds)

Table 37: BLE Communication test for NoiseSentinel

7.3.5 Performance Observations and Optimisations

The following optimisations were applied during development based on performance observations:

- Server-side pagination has been implemented for all list endpoints (challans, FIRs, court cases, users) with a maximum of 50 records per page, to avoid excessive data transfer and browser memory pressure for large data sets.
- Image compression is done on the mobile device using expo-image-manipulator, which reduces an average payload of around 4–6MB (raw camera output) to under 2MB before upload, saving significant upload time on mobile networks.
- BLE connection termination after test completion helps conserve the battery of the Android device and avoids the BLE stack maintaining an idle connection that could affect subsequent device scans.
- Swagger UI is disabled in the production Docker build, reducing middleware overhead in the production environment.
- All three containers have restart policies of unless-stopped, so they will automatically restart on transient failures without requiring human intervention.

Chapter 8:

Conclusion

Introduction

This last chapter provides a summary of the results of the NoiseSentinel project, following on from a reflection of the objectives reached, the technical milestones achieved and an honest assessment of the system's current capabilities and limitations. It takes a look at the lessons learned in software engineering practices during the development and provides some strategic recommendations for future enhancements that could make the platform a fully-featured smart enforcement system — one that can be deployed at the institutional level.

8.1 Achievements

The core of the NoiseSentinel project has been achieved in that the subjective, paper-based process of enforcing noise and emission limits on road vehicles has been replaced by an integrated, multi-platform digital process, in which evidence collected at the hardware level (by the sensor) is processed all the way to the judicial verdict in a single connected process.

The following are the major technical achievements of the project:

- **Full-stack multi-platform deployment.** The system has been deployed and is live in production at noisesentinel.tech and consists of four interconnected parts: a sensing device based on the ESP32 IoT, an Android mobile application built using React Native, an ASP.NET Core Web API, and a React web portal. All of them are talking to each other through a secure containerised infrastructure on DigitalOcean with Caddy automatically handling SSL encryption, Cloudflare handling DNS and edge security, Azure SQL Server as the production database, and SMTP2GO for transactional email sending. To provide a completed multi-component system in one academic year is a significant engineering accomplishment.
- **BLE IoT integration.** One of the most technically demanding parts of the project was the seamless integration of the ESP32 device and the mobile application developed using the React Native framework, using Bluetooth Low Energy (BLE) with a structured JSON command and response protocol over the GATT characteristics of the device. It implements both noise and emission tests (ten seconds and forty seconds respectively, including real time progress indicators), graceful disconnection handling, and device state management via LED indicators, all reliably on the physical device.

- **Dual-domain violation enforcement.** The platform marks the first of its kind in the enforcement landscape of Pakistan to process violations of noise limits (dBA measurement) and vehicular emission violations (CO, HC, NOx, CO2 measurement) in one device, one application and one workflow. The on-device classification engine compares readings to various severity thresholds and sends a classification label with the sensor result.
- **Full 6-role legal workflow.** The system features a fully fledged enforcement and judicial process, utilising six different user roles: Administrator, Police Officer, Station Authority, Court Authority, Judge and Public, and enforces the roles strictly through JWT on all API endpoint calls. The workflows span from the field challan to the Station Authorities to the FIR to the court case hearing to the verdict, and all records are permanently locked once the case is closed.

8.2 Critical Review

A fair assessment of the NoiseSentinel platform finds a complete and robust system with some known limitations that should be openly disclosed.

8.2.1 Strengths

- **Objective evidence capture.** The platform removes the biggest gap in the present enforcement — the manual data entry point between the instrument and the enforcement record — by collecting data directly from the sensor by means of calibrated hardware and sending it straight to the enforcement record via BLE
- **Connected evidence chain.** The linking of EmissionReport, Challan, FIR, CourtCase and Verdict records in the database makes the full evidence trail traceable from one case record. There is no paper-based handoff between any steps in the process.
- **Live production deployment.** The system is not a prototype and is not executed on a local machine. It runs on a real domain and is served via a real reverse proxy with real TLS certificates, and has a cloud-managed database. This is a higher level of engineering completion than is usually seen in academic final year projects.

8.2.2 Limitations

- **Cryptographic signing is not implemented.** The biggest difference between the designed specification and the delivered system is related to the digital signature of Emission Reports. The firmware sends a "signature": "SHA256_PLACEHOLDER" field in all responses, and no real cryptographic signing is done. In version 1.0 the emission report data is thus not cryptographically tamper evident. This undermines the legal defensibility of the evidence chain and needs to be fixed in future iterations for institutional rollout.
- **Offline mode not developed.** All challan submission and data retrieval operations on the mobile application require network connectivity. When there is poor or no cellular connectivity, officers cannot issue challans in that area until connectivity is restored. This is a recognised limitation and is considered a future enhancement.
- **NOx and CO2 are not measured directly, but estimated.** The hardware does not have an integrated NOx sensor. The NOx reading is estimated at 30% of the HC reading and CO2 is calculated from a CO ratio. These estimates are documented but would not hold up in actual legal proceedings that mandate direct measurement evidence.
- **Android only.** The existing mobile app is for Android only. There is no iOS support in version 1.0, which requires a separate native build process and App Store certification.
- **Battery monitoring is emulated.** A placeholder battery level value is provided by the firmware as hardware ADC-based battery monitoring is not implemented in version 1.0. This value is not suitable for field officers to use for planning purposes.

8.3 Lessons Learned

The development process for the NoiseSentinel platform, including four highly interdependent subsystems, offered many technical and project management lessons.

- **BLE management is significantly more complex than REST API integration.** Controlling the lifecycle of a BLE connection — scanning, connecting, subscribing to notifications, writing commands to the device, handling disconnected devices, recovering from failures — necessitated careful asynchronous state management that was more challenging than regular HTTP API calls. An additional friction point during development was the need for a native development build for the react-native-ble-plx library, which does not run in Expo Go.

- **The restrictions of Expo Go should be established in advance of mobile development.** A delay occurred with IoT integration since BLE functionality was not available in Expo Go but required a native development build. For future projects that require native device capabilities, this constraint should be discovered and the development environment set up accordingly before attempting integration.
- **Clean Architecture benefits during the integration process.** The separation of Web API, BLL and DAL layers ensured that when changes to the database schema were needed — such as adding BankSettings configuration, the public OTP store and the EmissionReport-to-Challan linkage — they were made only in the DAL and service classes without breaking API contracts or cascading into controllers. This confirmed the value of the architectural investment made at the beginning of the project.
- **Security needs to be built from the ground up on the first sprint.** For a system intended for use by legal and judicial entities, JWT authentication, password hashing, role enforcement on each endpoint, and secure credential management via environment variables must all be in place as base infrastructure before any feature work begins. Retrofitting these controls at a later stage would have been significantly more difficult and costly.
- **Docker Compose takes production deployment to a new level.** All three backend services were containerised from the outset of the project, meaning the deployment from local development to production on DigitalOcean involved only environment configuration and a docker compose up -d --build. There were no environment-specific dependency conflicts, no manual server configuration and no differences between development and production environments.

8.4 Future Enhancements and Recommendations

The following improvements are suggested for the next versions of the NoiseSentinel platform, listed in order of priority:

8.4.1 High Priority

- **Cryptographic signing of Emission Reports.** The most pressing improvement needed to enable institutional deployment is the use of real digital signatures for emission report data. A signature field already exists in the firmware's JSON response. The preferred

approach is firmware-level HMAC-SHA256 signing with a secret key provisioned at IoT device registration time, with the backend verifying the signature on ingestion. This would render emission report data cryptographically tamper evident at the time of capture.

- **Payment gateway integration.** Replacing the current manual bank transfer confirmation with a real payment gateway — JazzCash, Easypaisa or a Bank API integration — would eliminate the weaknesses of the trust-based payment model. Payment confirmation would be automatic upon completion, eliminating the possibility of false payment claims. This is essential for making the public payment module a viable revenue collection mechanism.

8.4.2 Medium Priority

- **GPS geolocalisation of violations.** Incorporating GPS coordinates with the challan at the time of generation would provide exact location evidence of the violation. The React Native application can access device GPS via Expo Location. This information would strengthen the legal record and enable hotspot analysis for enforcement planning.
- **Direct NOx sensor integration.** With an estimated NOx calculation (30% of HC), results are not directly measured and would be less defensible in formal legal proceedings than readings from a dedicated electrochemical NOx sensor such as the MICS-2714. This would require a hardware revision to the ESP32 device and an update of the sensor calibration library.

8.4.3 Long-Term Recommendations

- **Machine learning audio classification.** An ML model implemented on the ESP32 firmware or in the mobile app via TensorFlow Lite could differentiate vehicle exhaust sounds from other ambient noises such as wind, construction or passing sirens. This would decrease false positive readings and enhance the scientific defensibility of noise violation evidence.
- **Automated license plate recognition.** Integrating a computer vision library into the mobile application's image capture process to directly recognise vehicle registration numbers from captured images would eliminate the manual plate entry step, reduce transcription errors and speed up challan generation. This could be achieved with the Google ML Kit text recognition API, which is supported by React Native.

- **Dynamic threshold configuration.** The current noise and emission classification thresholds are hard-coded in the ESP32 firmware and can only be changed by firmware update. Shifting threshold definitions to the backend, stored in a configurable rules table and sent to the device at connection, would allow administrators to adjust thresholds for different vehicle classes, geographic areas or regulatory requirements without redeploying firmware.

8.5 Concluding Remarks

NoiseSentinel showcases the technical viability of a complete end-to-end vehicle enforcement platform — from custom IoT hardware, to cross-platform mobile application, cloud backend API and multi-role web portal — all within a university final year project. It is not a simulation or mock-up system. It is a deployed, working platform available at a real domain, utilising real cloud infrastructure and real sensor hardware with a real BLE protocol.

The fundamental concept of this project is that the hardware-software separation which is inherent in the current enforcement approach — where sensor data and legally relevant documents are separate entities which are manually linked — can be overcome by careful system integration. The data the ESP32 device produces when a police officer starts an emission test is the same data that ends up in the challan record generated in the Azure SQL database, and not two distinct records sharing the same number.

There are limitations to the system that are acknowledged in this chapter: the lack of payment verification, the NOx estimates, the absence of offline mode, and the unimplemented emission report signing. These describe the distance from the current prototype to a system that is ready for full institutional deployment. Section 8.4's roadmap outlines a prioritised path to closing that gap. That work would be built upon a solid, tested, and live foundation.

Appendices

Appendix A: Promotional and Exhibition Material

This appendix includes the promotional material created for the NoiseSentinel platform for the Final Year Project Exhibition and public presentation. The following materials were created to convey the purpose, main features, and impact of the system to faculty evaluators, industry visitors, and other students.

A.1 Project Overview Poster

The project poster offers a high-level, visual overview of the NoiseSentinel system. It introduces the problem statement — the need for manual noise and emission enforcement in Pakistan — and depicts the simplified system architecture diagram consisting of four interconnected components: ESP32 IoT device, mobile application, Web API, and web portal. The key statistics highlighted are the legal noise limit of 85 dBA, the 4 gases monitored (CO, HC, NOx, CO2), and the 6 user roles supported by the platform. The poster features a QR code that leads to the live deployment at noisesentinel.tech.

A.2 System Demonstration Flyer

A demonstration flyer was provided to FYP visitors. It presents an overview of the NoiseSentinel platform in an accessible way for a general audience, detailing the real-life problem, the system's solution and the path of a violation from detection to verdict in 5 steps:

1. Officer pairs ESP32 device with the mobile app via Bluetooth.
2. Noise or vehicle emissions are automatically measured by the device.
3. Officer issues a digital challan with photo evidence.
4. Station Authority reviews and escalates further if necessary.
5. The Judge examines the evidence and issues a verdict.

The flyer contains the names of the project team, the supervisor's name, department and university information.

A.3 Live Demonstration Set Up

For the FYP exhibition, a real-time demonstration of the whole enforcement process was set up:

- The ESP32 device was powered and visitors could observe the BLE pairing process in the Android mobile application.
- The noise test was demonstrated live with the dBA value and classification displayed in real time on the mobile application screen.
- All Station Authority, Court Authority and Judge dashboards, and the public case status page were displayed on a laptop browser through the web portal.
- The entire process of preparing a test challan on the mobile application and progressing it to a verdict on the web portal was showcased before the evaluators.

Appendix B: System Configuration Reference

This appendix is a reference for the environment variables and configuration keys needed to deploy the NoiseSentinel platform. This information is for use by system administrators and deployment engineers.

B.1 Docker Compose Environment Variables

These environment variables should be set in the .env file located in the root of the repository before running docker compose up.

Variable	Description	Example
DB_CONNECTION_STRING	Azure SQL Server connection string	Server=tcp:server.database.windows.net;Database=NoiseSentinel;...
JWT_EXPIRATION_MINUTES	Token validity in minutes	1440
SMTP_HOST	SMTP relay hostname	mail.smtp2go.com
SMTP_PORT	SMTP relay port	587
SMTP_SENDER_EMAIL	Sender email address	admin@noisesentinel.tech
SMTP_SENDER_NAME	Sender display name	NoiseSentinel

References and Bibliography

[1] World Health Organization, Environmental Noise Guidelines for the European Region, WHO Regional Office for Europe, Copenhagen, Denmark, 2018

[2] Government of Pakistan, Motor Vehicle Rules, 1969, Ministry of Communications, Islamabad, Pakistan, 1969 (with subsequent amendments).

[3] Government of Pakistan, Pakistan Environmental Protection Act, 1997, Ministry of Environment, Islamabad, Pakistan, 1997.

[4] Espressif Systems, ESP32 Technical Reference Manual, Version 5.1, Espressif Systems, Shanghai, China, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[5] Maxim Integrated, MAX9814 Microphone Amplifier with AGC and Low-Noise Microphone Bias Datasheet, Maxim Integrated Products, San Jose, CA, USA, 2011. [Online]. Available: https://www.maximintegrated.com/en/products/analog/audio/MAX9814.html

[6] Zhengzhou Winsen Electronics Technology, MQ-7 Sensitive Material CO Gas Sensor Datasheet, Winsen Electronics, Zhengzhou, China, 2014.

[7] Zhengzhou Winsen Electronics Technology, MQ-2 Sensitive Material LPG, i-butane, Propane, Methane, Alcohol, Hydrogen Gas Sensor Datasheet, Winsen Electronics, Zhengzhou, China, 2014.

[8] Bluetooth SIG, Bluetooth Core Specification Version 4.2, Bluetooth Special Interest Group, Kirkland, WA, USA, 2014. [Online]. Available: https://www.bluetooth.com/specifications/specs/core-specification-4-2/